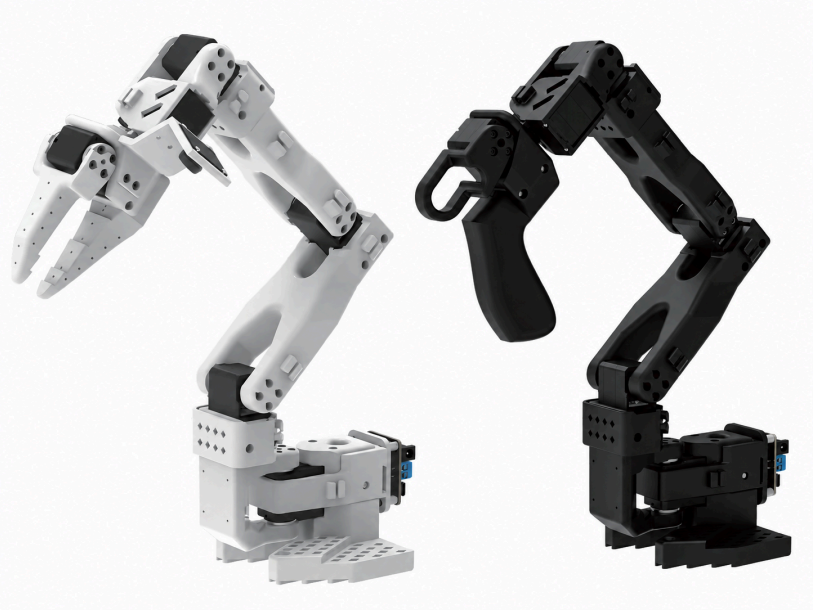
SO-ARM101开源6轴机械臂使用文档

1. LeRobot 简介

LeRobot 是由 **Hugging Face** 开发的开源机器人学习框架，主要用于机器人行为克隆和强化学习。它为研究人员和开发者提供了统一的平台，可用于训练、部署和评估机器人策略。**SO-ARM101** 是 LeRobot 生态中常用的开源机械臂，支持主从臂遥操作。用户可以通过主臂控制从臂完成任务，并采集运动轨迹数据，用于训练机器人自主执行抓取、搬运等操作。该机械臂结构轻量、装配友好，适合作为 LeRobot 学习和实验的入门硬件平台。



1.1 核心特性

1. **多模态数据支持**：支持视觉、触觉、音频等多种传感器数据
2. **灵活的策略架构**：支持行为克隆、强化学习等多种学习范式
3. **丰富的机器人支持**：兼容多种机器人平台和硬件配置
4. **云端训练支持**：支持分布式训练和云端部署
5. **易于扩展**：模块化设计，便于添加新的机器人类型和算法

1.2 技术架构

LeRobot 基于 **PyTorch** 构建，采用现代深度学习技术栈：

1. **数据处理**：高效的数据加载和预处理管道
2. **模型训练**：支持多种神经网络架构和训练策略
3. **实时推理**：优化的推理引擎，支持实时机器人控制
4. **可视化工具**：丰富的数据可视化和训练监控工具

1.3 生态系统

LeRobot 与 **Hugging Face** 生态系统深度集成：

1. **模型共享**：通过**Hugging Face Hub**分享训练好的模型
2. **数据集管理**：统一的数据集存储和版本控制
3. **社区支持**：活跃的开源社区和丰富的文档资源
4. **持续更新**：定期发布新功能和性能优化

1.4 机械臂介绍

SO-ARM101 是一款用于机器人学习、遥操作控制和动作数据采集的桌面级机械臂系统。该系统通常由 **主臂** 和 **从臂** 两部分组成，两者结构相似，但功能定位不同。主臂主要用于人工操作和动作示教，操作者通过拖动主臂完成指定动作，系统会实时采集各关节的位置信息；从臂作为执行端，根据主臂传递的动作数据同步运动，从而实现主从控制。

1.4.1 主臂介绍

SO-ARM101 的主臂也称为 **Leader Arm** 或 **遥操作臂**，主要用于人工示教和动作采集。操作人员可以直接用手拖动主臂，使其完成抓取、移动、放置等动作。系统会实时读取各关节的位置数据，并将这些数据传递给从臂执行，或保存为后续模型训练所需的数据集。

主臂使用 **HX-10HM 舵机** 作为关节驱动单元，其减速比为 **1:147**。相较于高减速比舵机，HX-10HM 的减速比较低，关节转动阻力相对较小，手动拖动时更加顺滑，适合用于人工操作和姿态采集。由于主臂主要承担“输入动作”的作用，并不需要直接搬运物体，因此更注重操作手感、响应速度和动作采集的流畅性。

舵机型号	转动速度	堵转扭矩	减速比	电压范围
HX-10HM	0.10 sec/60° @ 11.1V	10 kg·cm @ 11.1V	1:147	9–12.6V

1.4.2 从臂介绍

SO-ARM101 的从臂也称为 **Follower Arm** 或 **执行臂**，主要负责执行主臂传来的动作指令。在遥操作模式下，操作者移动主臂，从臂会实时跟随主臂的关节姿态并同步运动；在模型推理或策略执行阶段，从臂可以根据训练好的控制策略，自动完成抓取、移动、放置等任务。

从臂使用 **HX-30HM 舵机** 作为关节驱动单元，其减速比为 **1:345**。与主臂所使用的 HX-10HM 舵机相比，HX-30HM 的减速比更高，在相同电机条件下可以获得更大的输出扭矩，关节承载能力和抗外力能力更强，更适合用于实际执行端的运动控制和物体抓取。

舵机型号	转动速度	堵转扭矩	减速比	电压范围
HX-30HM	0.19 sec/60° @ 11.1V	30 kg·cm @ 11.1V	1:345	9–12.6V

1.5 推荐学习路径

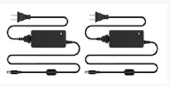
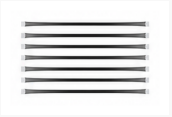
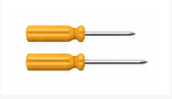
首次使用建议不要直接开始数据采集或模型训练。先完成环境配置和机械臂组装，再依次进行端口确认、主从臂校准及无视觉遥操作；确认机械臂运动稳定后，再接入摄像头进行数据采集。训练和推理应使用同一套摄像头配置与任务场景，这样更容易定位问题并获得稳定效果。

2. 硬件与环境准备

2.1 硬件清单

::: DIY 版（散件）

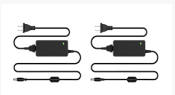
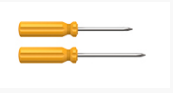
> **Note**
>
> **此套餐不含 3D 打印件，需要自行将机械臂的支架打印出来，3D打印图纸可在资料附录中找到，打印机可参考以下参数进行打印：**
>
> - **材料：PLA**
> - **层高：0.20mm**
> - **填充密度：15%**

序号	名称	图片	数量	序号	名称	图片	数量
1	HX-30HM 总线舵机		6	2	HX-10HM 总线舵机		6
3	多功能驱动模块		2	4	12V 5A 适配器（DC5.5x2.5）		2
5	USB 集线器（4 孔，长 1000mm）		1	6	2 寸 G 字夹		4
7	Type-C 数据线（长 1000mm）		2	8	拍摄支架		1
9	舵机线（长 100mm）		3	10	舵机线（长 160mm）		5
11	舵机线（长 200mm）		7	12	金属主舵盘		14
13	金属副舵盘		12	14	扎带（3x150mm）		10
15	螺丝刀		2	16	M2.5x10 双通尼龙柱		10
17	M3x10 沉头螺丝		4	18	M3x5x7 带垫自攻螺丝		14
19	M3x6 螺丝		120	20	M2x6 自攻螺丝		40
21	M2.5x4 螺丝		20	22	M2x28 半牙平尾自攻螺丝		28

序号	名称	图片	数量	序号	名称	图片	数量
23	M3 螺母		4				

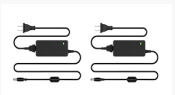
:::

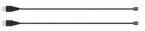
::: 入门版（成品）

序号	名称	图片	数量	序号	名称	图片	数量
1	主臂		1	2	从臂		1
3	12V 5A 适配器（DC5.5x2.5）		2	4	USB 集线器（4 孔，长 1000mm）		1
5	2 寸 G 字夹		4	6	Type-C 数据线（长 1000mm）		2
7	舵机线（长 100mm）		1	8	舵机线（长 160mm）		1
9	舵机线（长 200mm）		1	10	金属主舵盘x2 + 副舵盘x2		1 组
11	扎带（3x150mm）		10	12	螺丝刀		2
13	螺丝螺母配件包		1				

:::

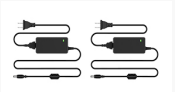
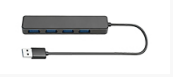
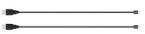
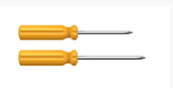
::: 标准版（成品）

序号	名称	图片	数量	序号	名称	图片	数量
1	主臂		1	2	从臂		1
3	12V 5A 适配器（DC5.5x2.5）		2	4	USB 集线器（4 孔，长 1000mm）		1

序号	名称	图片	数量	序号	名称	图片	数量
5	2 寸 G 字夹		4	6	Type-C 数据线（长 1000mm）		2
7	舵机线（长 100mm）		1	8	舵机线（长 160mm）		1
9	舵机线（长 200mm）		1	10	金属主舵盘x2 + 副舵盘x2		1 组
11	扎带（3x150mm）		10	12	螺丝刀		2
13	螺丝螺母配件包		1	14	30W 像素高清广角摄像头		1
15	摄像头支架		1	16	USB 数据线（长 1000mm）		1

:::

::: 豪华版（成品）

序号	名称	图片	数量	序号	名称	图片	数量
1	主臂		1	2	从臂		1
3	12V 5A 适配器（DC5.5x2.5）		2	4	USB 集线器（4 孔，长 1000mm）		1
5	2 寸 G 字夹		4	6	Type-C 数据线（长 1000mm）		2
7	舵机线（长 100mm）		1	8	舵机线（长 160mm）		1
9	舵机线（长 200mm）		1	10	金属主舵盘x2 + 副舵盘x2		1 组
11	扎带（3x150mm）		10	12	螺丝刀		2

序号	名称	图片	数量	序号	名称	图片	数量
13	螺丝螺母配件包		1	14	拍摄支架		1
15	30W 像素高清广角摄像头		1	16	200W 像素高清广角摄像头		1
17	摄像头固定件		1	18	摄像头支架		1
19	USB 数据线（长 1000mm）		2				

:::

2.2 环境配置

> **Note**
>
> **本教程会与官方文档保持同步更新。若遇到环境或软件问题，请优先查阅文末的常见问题解答；若仍未解决，欢迎联系客服加入交流群获取帮助。**

环境配置分为两部分：先安装 Miniconda 并完成换源，再创建 `lerobot` 虚拟环境并安装项目依赖。Windows 和 Ubuntu 的操作入口不同，但后续都需要进入同一个 `lerobot` 环境执行命令。

2.2.1 安装 Miniconda

:::: Windows 系统

- **Windows 系统安装**

①下载 **Miniconda** 安装包

Miniconda 官网安装包

找到 **Miniconda3-py311_25.7.0-2-Windows-x86_64.exe**，将安装包下载到电脑上。（也可以直接使用：`软件工具&源码程序\软件工具`）

Miniconda 安装包下载通常需要 1～10 分钟，具体取决于网络带宽；若下载中断或速度长期过慢，可改用下方的镜像站链接。

```
Miniconda3-py311_25.7.0-2-Windows-x86_64.exe
```

> **Note**
>
> **由于官网是国外网站，国内访问速度可能较慢，推荐使用清华大学镜像源下载。**

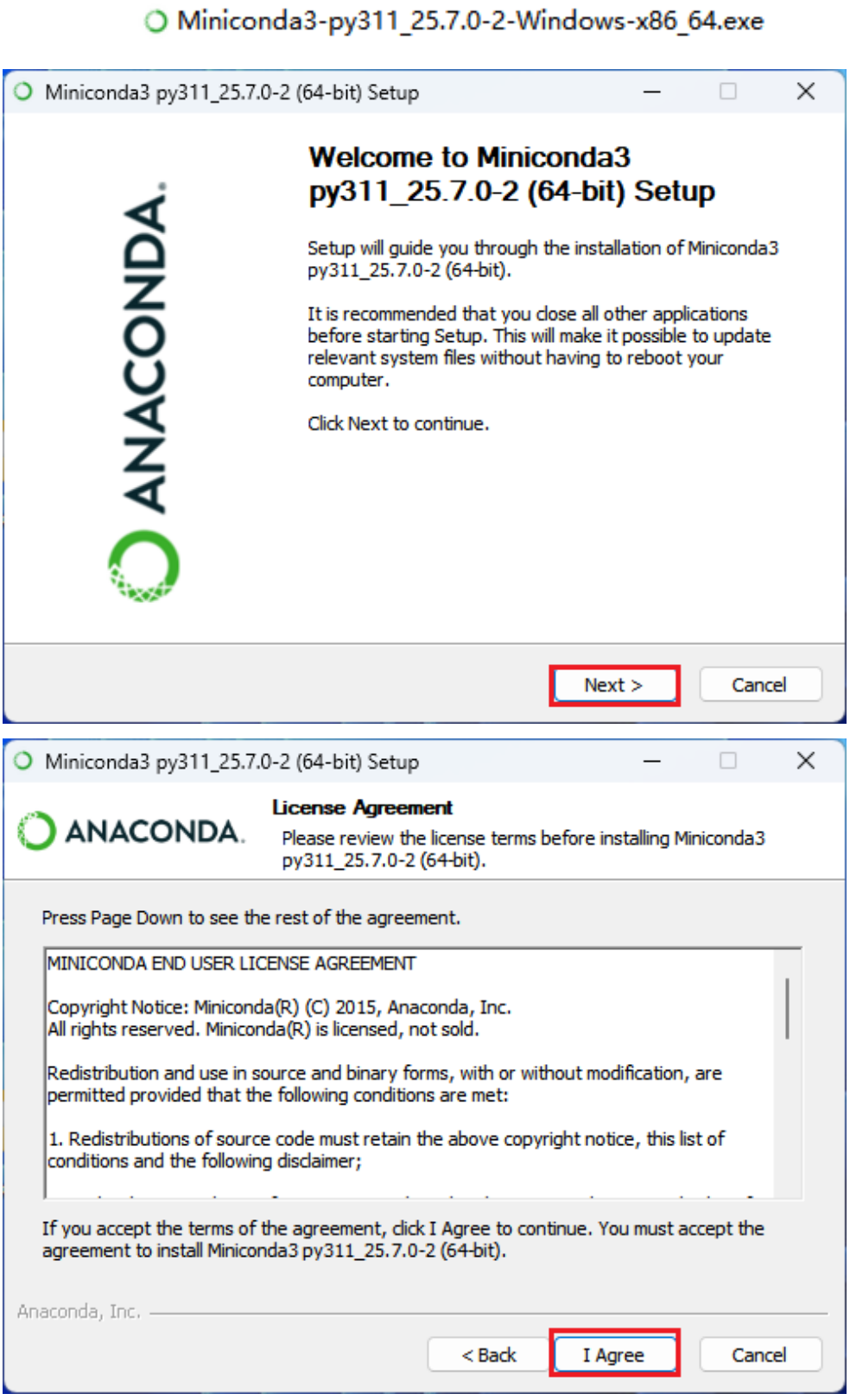
Index of /anaconda/miniconda/ | 清华大学开源软件镜像站 | Tsinghua Open Source Mirror

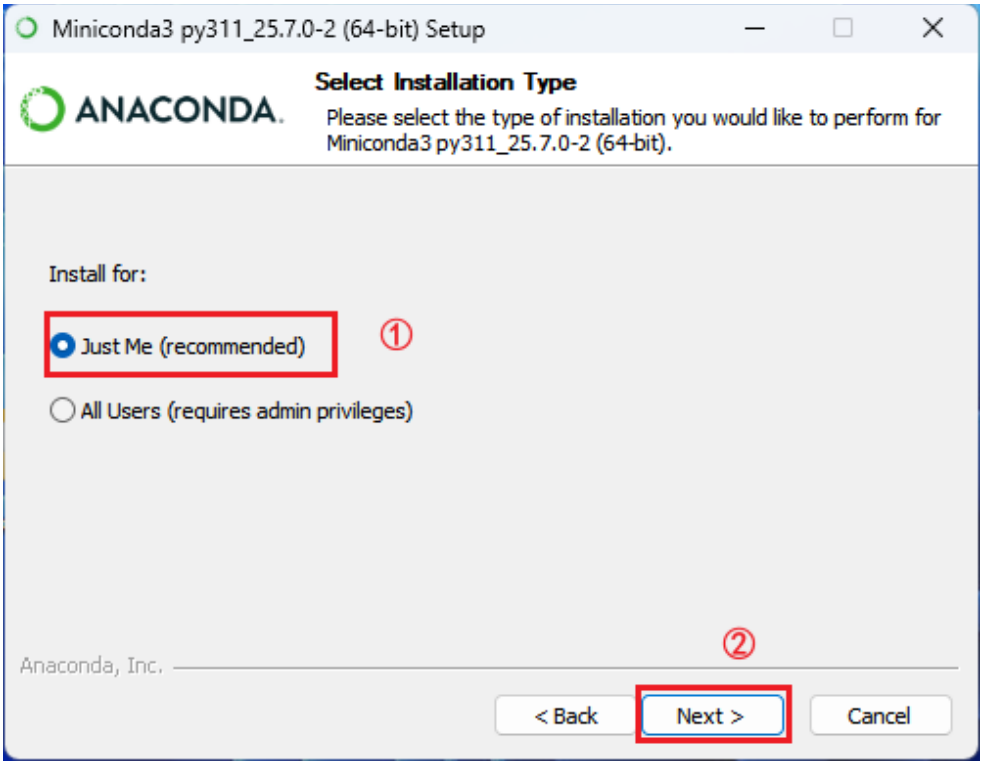
打开上方链接下载，这里同样选择 `Miniconda3-py311_25.7.0-2-Windows-x86_64.exe`。

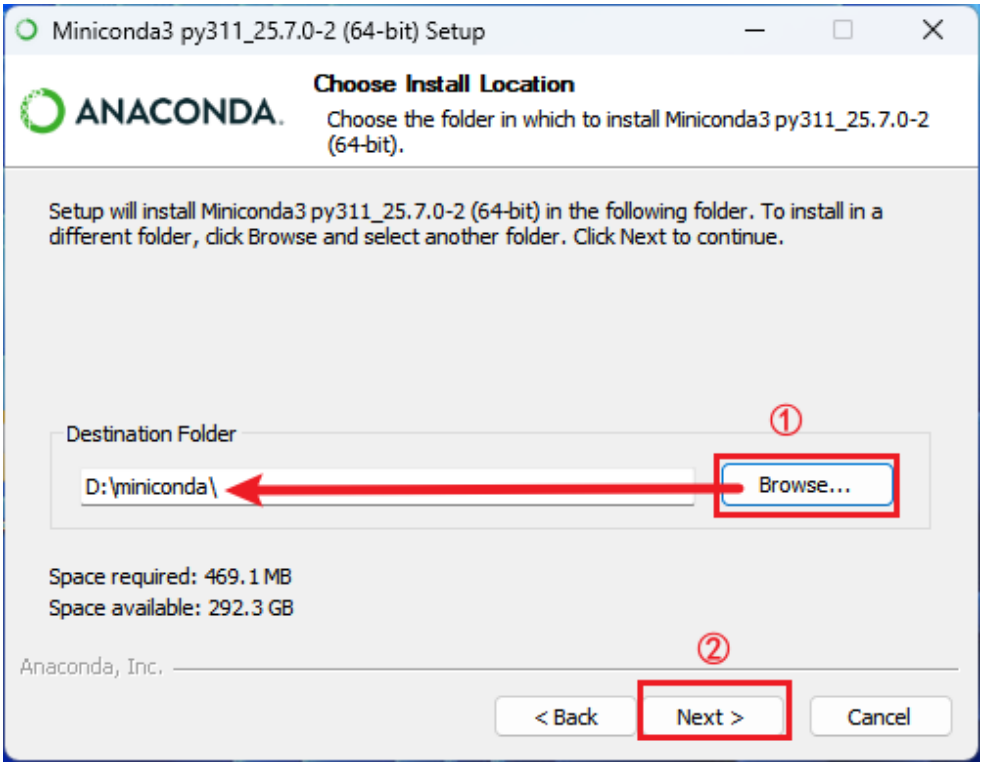
```
Miniconda3-py311_25.7.0-2-Windows-x86_64.exe
```

②安装 **Miniconda**

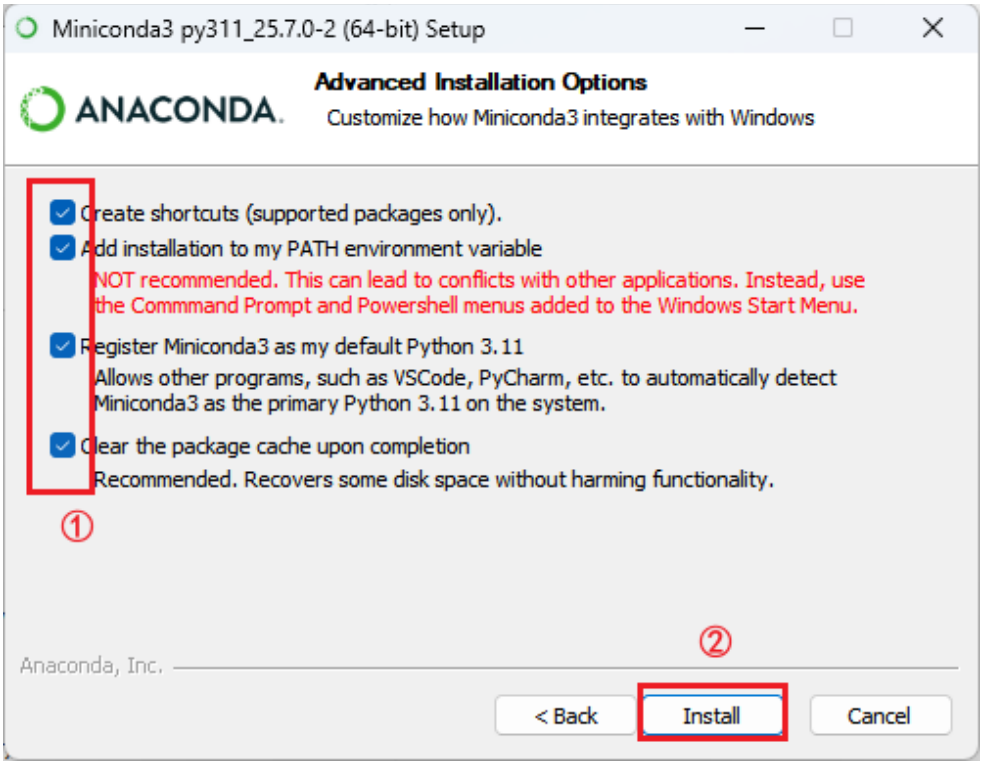
找到下载好的 **Miniconda** 安装包，双击安装。

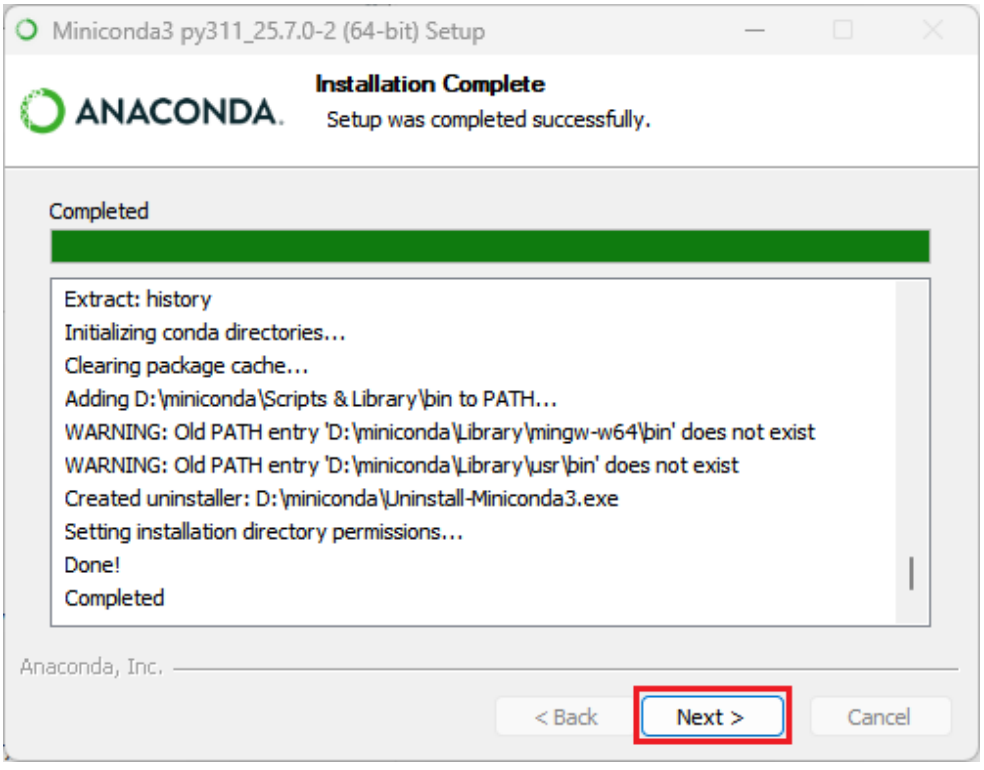


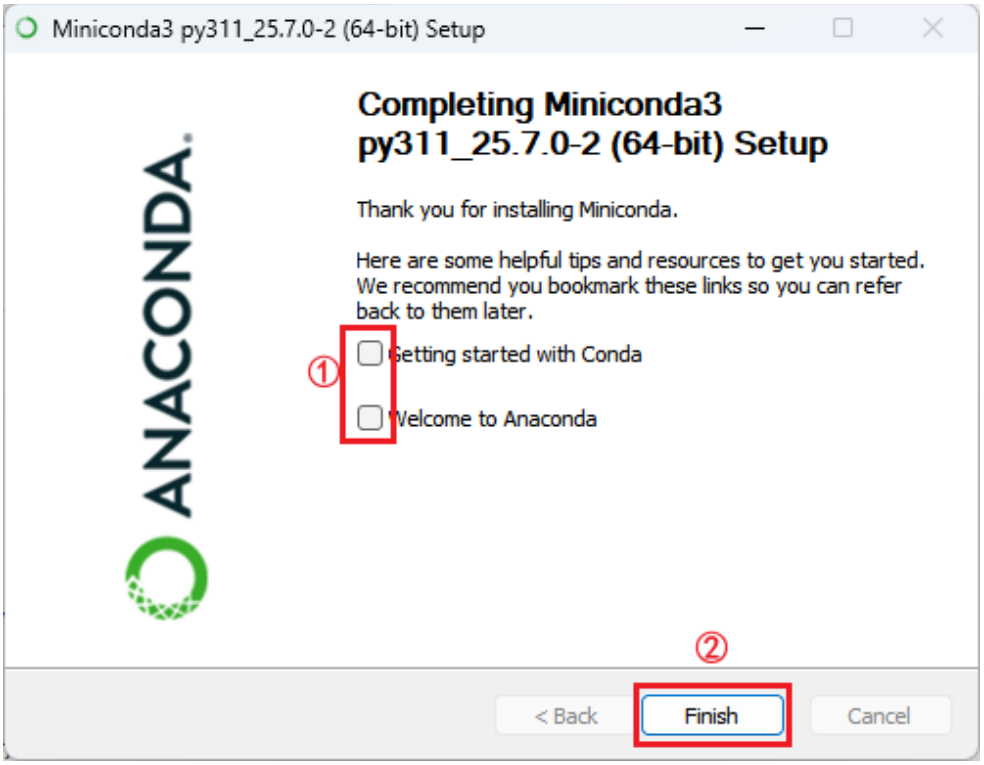




下方一定要全部勾选，否则环境配置会出现问题！


Miniconda3 py311_25.7.0-2 (64-bit) Setup
ANACONDA.
Advanced Installation Options
Customize how Miniconda3 integrates with Windows
Create shortcuts (supported packages only).
Add installation to my PATH environment variable
NOT recommended. This can lead to conflicts with other applications. Instead, use the Commmand Prompt and Powershell menus added to the Windows Start Menu.
Register Miniconda3 as my default Python 3.11
Allows other programs, such as VSCode, PyCharm, etc. to automatically detect Miniconda3 as the primary Python 3.11 on the system.
Clear the package cache upon completion
Recommended. Recovers some disk space without harming functionality.
①
②
Anaconda, Inc.
< Back
Install
Cancel


Miniconda3 py311_25.7.0-2 (64-bit) Setup
ANACONDA.
Installation Complete
Setup was completed successfully.
Completed
Extract: history
Initializing conda directories...
Clearing package cache...
Adding D:\miniconda\Scripts & Library\bin to PATH...
WARNING: Old PATH entry 'D:\miniconda\Library\mingw-w64\bin' does not exist
WARNING: Old PATH entry 'D:\miniconda\Library\usr\bin' does not exist
Created uninstaller: D:\miniconda\Uninstall-Miniconda3.exe
Setting installation directory permissions...
Done!
Completed
Anaconda, Inc.
< Back
Next >
Cancel



③换源

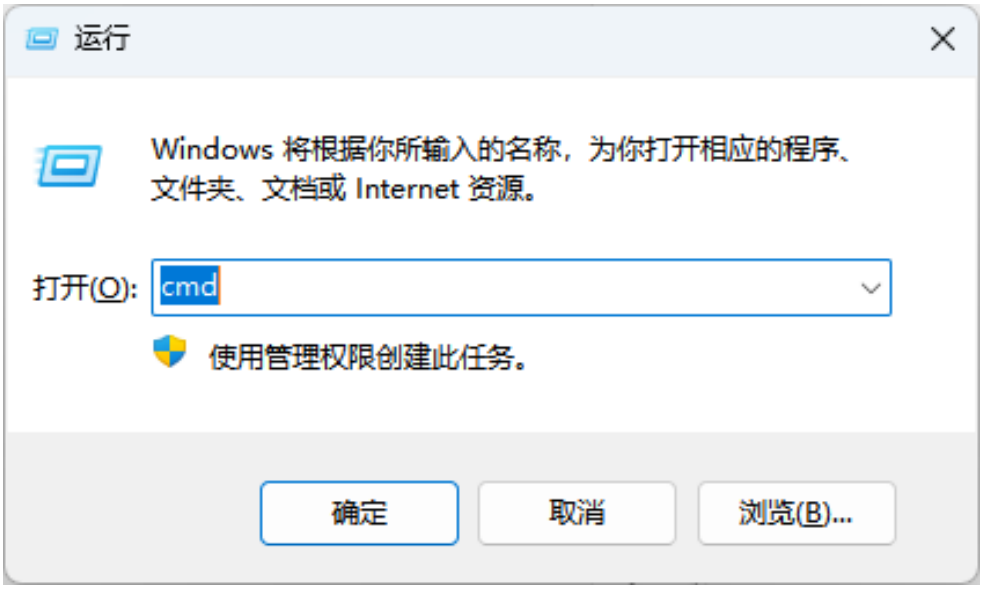
> **Note**
>
> **为什么需要换源：** Conda 默认从海外软件仓库下载软件包和索引信息，国内网络环境下可能出现下载速度慢、连接超时或依赖解析失败。将下载地址切换到镜像站后，安装的软件包和版本并不会改变，只是使用更稳定、速度更快的下载节点。若所在网络无法访问镜像站，可将 `.condarc` 恢复为默认配置后再尝试。

[anaconda | 镜像站使用帮助 | 清华大学开源软件镜像站 | Tsinghua Open Source Mirror]

点击链接进入 **Miniconda** 软件仓库，找到下图框选的第三方源。

```
channels:
  - defaults
show_channel_urls: true
default_channels:
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/r
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/msys2
custom_channels:
  conda-forge: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
  pytorch: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
```

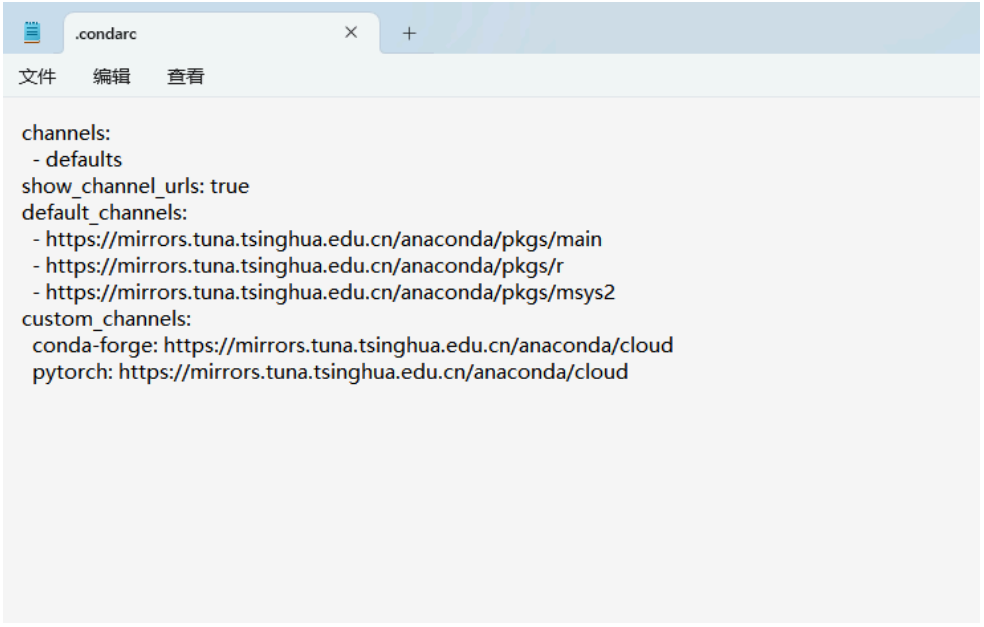
按 **Win+R**，输入 **cmd**，打开终端。



在终端中输入指令，生成 `.condarc` 文件。

```
conda config --set show_channel_urls yes
```

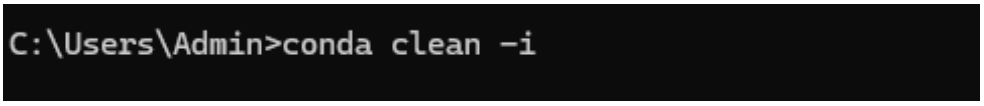
`.condarc` 文件一般位于用户目录下，例如 `C:\Users\Admin`。打开 `.condarc` 后，将内容替换为下图所示配置：



运行指令，清理索引缓存。

```
conda clean -i
```

该命令会删除 Conda 已缓存的软件包索引，让下一次安装或创建环境时重新读取刚配置的镜像地址。执行通常只需几秒到 1 分钟；首次重新读取索引时，后续命令可能会比平时稍慢。



::::

:::: Ubuntu 系统

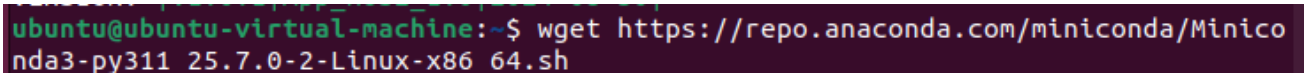
- **Ubuntu 系统安装**

①下载 **Miniconda** 安装包

按 **Ctrl+Alt+T** 打开终端，输入指令下载 **Miniconda** 安装包。

```
wget https://repo.anaconda.com/miniconda/Miniconda3-py311_25.7.0-2-Linux-x86_64.sh
```

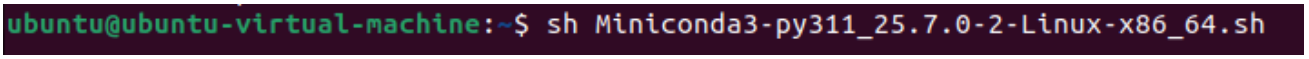
下载时间通常为 1～10 分钟。若下载速度长期较慢，可先完成本小节的换源配置，再重新尝试下载或使用课程资料中的安装包。



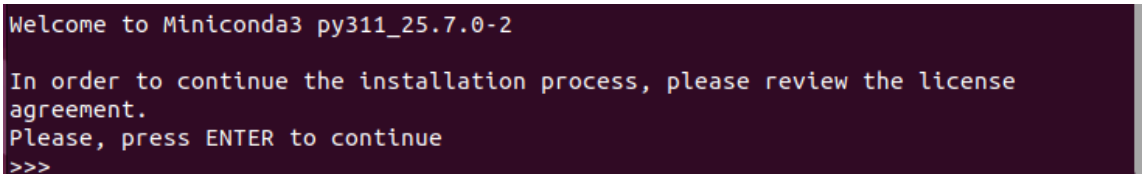
②安装 **Miniconda**

输入指令安装 **Miniconda**。

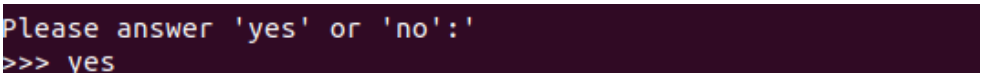
```
sh Miniconda3-py311_25.7.0-2-Linux-x86_64.sh
```



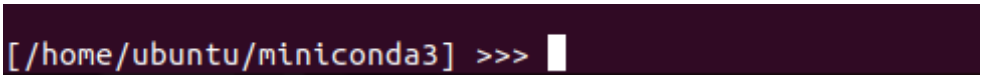
按下 **Enter** 继续。



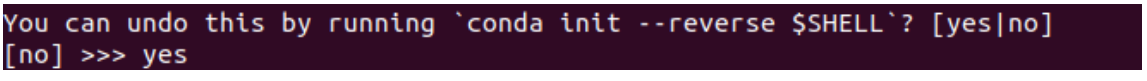
输入 **yes**，按下 **Enter** 继续。



这里默认安装到 `/home/ubuntu/miniconda3`。如果不需要更改安装目录，直接按下 **Enter** 继续即可；如需更改安装目录，输入目标路径后再按下 **Enter**。



输入 **yes** 后按下 **Enter**，即可完成安装。



安装完成后关闭终端，再重新打开终端，即可看到下图所示界面。

如果不想默认启用 **Miniconda** 的 **Python** 环境，需要在 `.bashrc` 文件中添加一行指令。

输入指令，打开 `.bashrc` 文件。

```
gedit .bashrc
```

```
(base) ubuntu@ubuntu-virtual-machine:~$ gedit .bashrc
```

找到 conda initialize 代码块，在其下方添加一行指令。添加完成后按 **Ctrl+s** 保存并关闭文件，重新打开终端后即可生效。

```
conda config --set auto_activate false
```

```
# >>> conda initialize >>>
# !! Contents within this block are managed by 'conda init' !!
__conda_setup="$('/home/ubuntu/miniconda3/bin/conda' 'shell.bash' 'hook' 2> /dev/null)"
if [ $? -eq 0 ]; then
    eval "$__conda_setup"
else
    if [ -f "/home/ubuntu/miniconda3/etc/profile.d/conda.sh" ]; then
        . "/home/ubuntu/miniconda3/etc/profile.d/conda.sh"
    else
        export PATH="/home/ubuntu/miniconda3/bin:$PATH"
    fi
fi

unset __conda_setup
# <<< conda initialize <<<

conda config --set auto_activate false
```

③换源

> **Note**
>
> **为什么需要换源：** Ubuntu 下的 Conda 同样默认访问海外软件仓库。配置镜像站只改变软件下载地址，不会改变安装包的功能或版本；完成配置后，后续创建环境和安装依赖通常会更稳定。

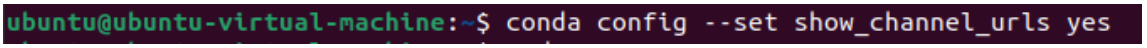
anaconda | 镜像站使用帮助 | 清华大学开源软件镜像站 | Tsinghua Open Source Mirror

点击链接进入 **Miniconda** 软件仓库，找到下图框选的第三方源。

```
channels:
  - defaults
show_channel_urls: true
default_channels:
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/r
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/msys2
custom_channels:
  conda-forge: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
  pytorch: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
```

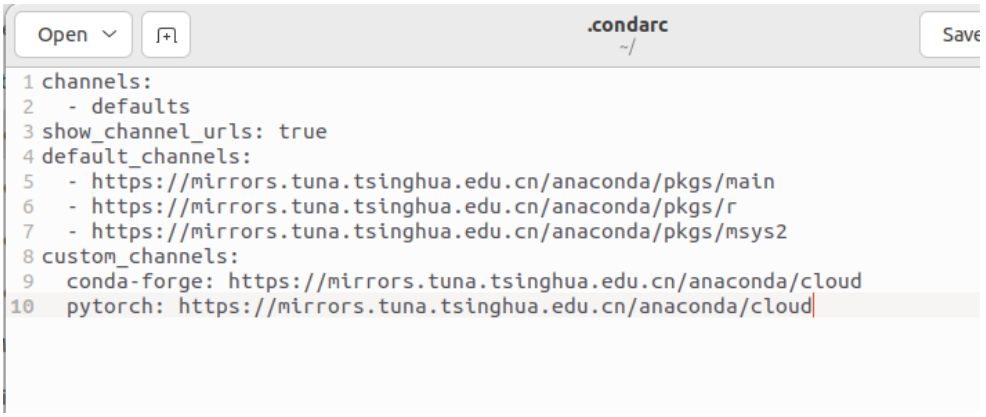
在终端中输入指令，生成 `.condarc` 文件。

```
conda config --set show_channel_urls yes
```



`.condarc` 文件一般位于用户目录下，例如 `/home/ubuntu`。使用 `gedit` 工具打开 `.condarc`，并将内容替换为下图所示配置。

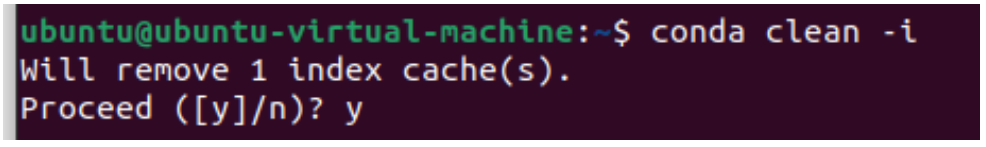
```
gedit .condarc
```



运行指令清除索引缓存，输入 **y** 后按下 **Enter** 即可。

```
conda clean -i
```

清除缓存是为了让 Conda 在下一次安装时重新加载新的镜像索引，通常只需几秒到 1 分钟。



::::

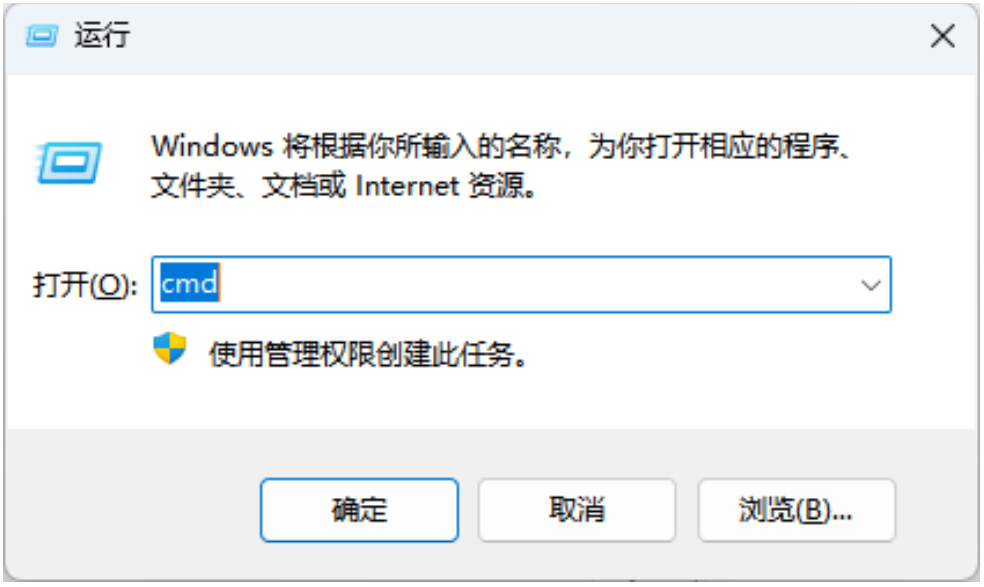
2.2.2 配置虚拟环境

:::: Windows 系统

- **Windows 配置虚拟环境**

①打开终端

按 **Win+R**，输入 **cmd**，打开终端。



②创建虚拟环境

输入指令并按下 **Enter**，创建虚拟环境并安装 **ffmpeg** 包（建议使用 7.1.1，支持 **libsvtav1 编码**）。

```
conda create -n lerobot python=3.10.18 ffmpeg=7.1.1 -c conda-forge
```

> **Note**
>
> **首次创建环境通常需要约 5～15 分钟。** 其中 Conda 解析依赖可能需要 1～5 分钟，随后会下载并安装 Python、ffmpeg 等软件包。网络较慢、首次使用镜像站或磁盘性能较低时，等待时间可能延长至 20 分钟左右；请勿在安装过程中关闭终端。

```
C:\Users\Admin>conda create -n lerobot python=3.10.18 ffmpeg=7.1.1 -c conda-forge
```

出现下图提示后，按 **y**，再按 **Enter** 继续。

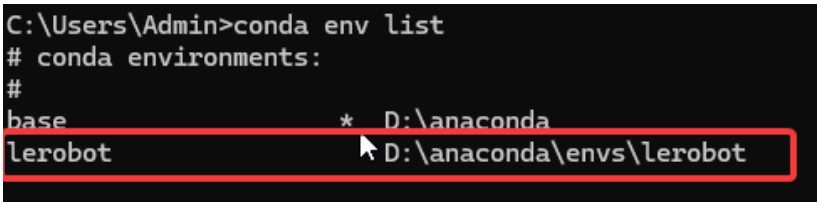
```
The following NEW packages will be INSTALLED:

    bzip2:           1.0.8-h2bbff1b_6        defaults
    ca-certificates: 2025.7.15-haa95532_0    defaults
    expat:           2.7.1-h8ddb27b_0        defaults
    libffi:          3.4.4-hd77b12b_1        defaults
    openssl:         3.0.17-h35632f6_0       defaults
    pip:             25.2-pyhc872135_0       defaults
    python:          3.10.18-h981015d_0      defaults
    setuptools:      78.1.1-py310haa95532_0  defaults
    sqlite:          3.50.2-hda9a48d_1       defaults
    tk:              8.6.15-hf199647_0       defaults
    tzdata:          2025b-h04d1e81_0        defaults
    ucrt:            10.0.22621.0-haa95532_0 defaults
    vc:              14.3-h2df5915_10        defaults
    vc14_runtime:    14.44.35208-h4927774_10 defaults
    vs2015_runtime:  14.44.35208-ha6b5a95_10 defaults
    wheel:           0.45.1-py310haa95532_0  defaults
    xz:              5.6.4-h4754444_1        defaults
    zlib:            1.2.13-h8cc25b3_1       defaults

Proceed ([y]/n)? y
```

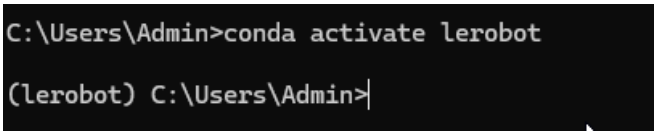
创建完成后，输入指令即可查看已创建的虚拟环境。

```
conda env list
```



③进入虚拟环境

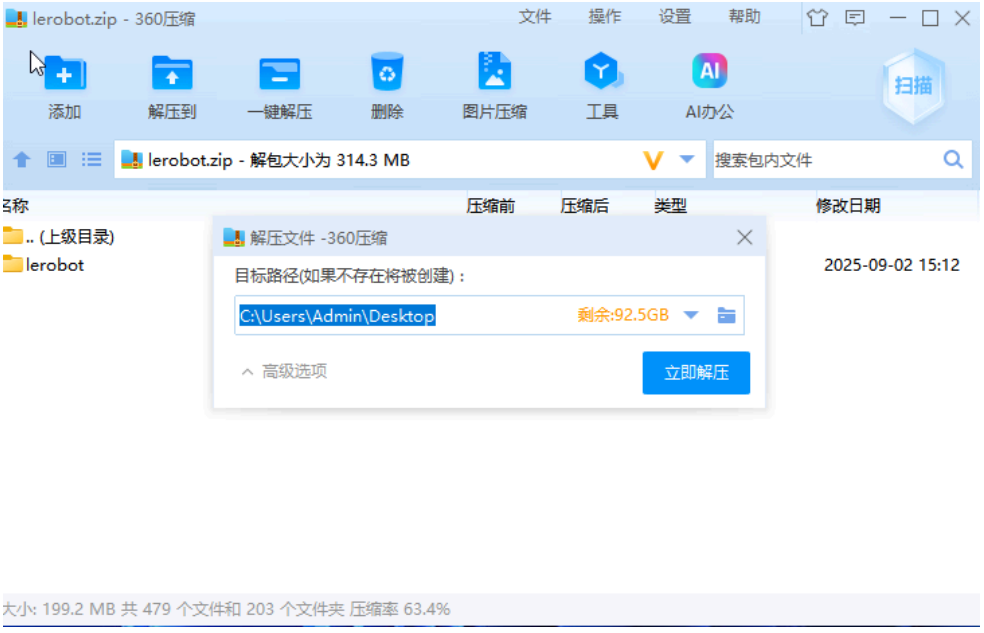
```
conda activate lerobot
```



④下载代码库

SO-ARM101 开源 6 轴机械臂工程包路径：`软件工具&源码程序\源码程序\lerobot.zip`

打开 **SO-ARM101 开源 6 轴机械臂** 工程包，将工程文件解压到桌面。

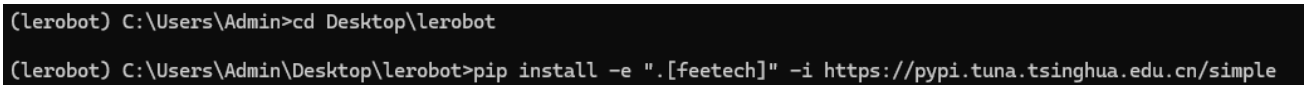


⑤安装依赖包

进入 **lerobot** 目录，安装 **lerobot** 依赖包，并指定添加舵机相关驱动。

```
cd Desktop\lerobot
pip install -e ".[feetech]" -i https://pypi.tuna.tsinghua.edu.cn/simple
```

首次安装舵机驱动及项目依赖通常需要约 3～10 分钟；如出现下载较慢，可先确认网络连接和镜像源配置，安装完成前不要关闭终端。



::::

:::: Ubuntu 系统

- **Ubuntu 配置虚拟环境**

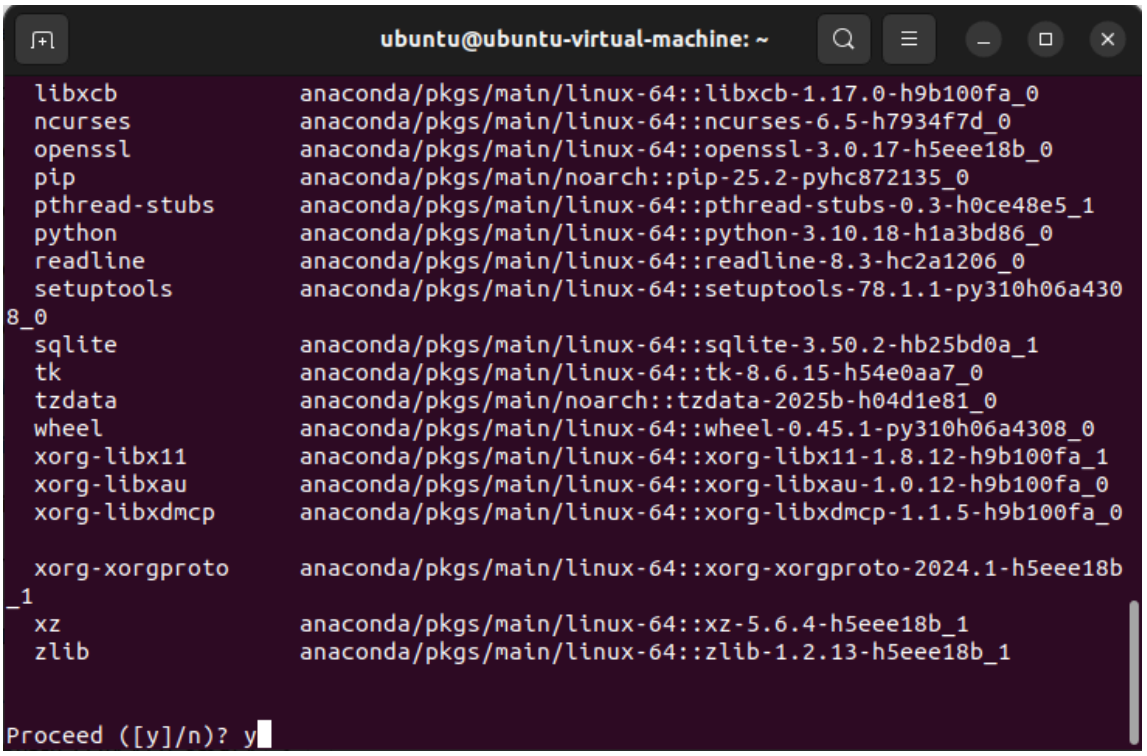
①创建虚拟环境

按 **Ctrl+Alt+T** 打开终端，输入指令并按下 **Enter**，创建虚拟环境并安装 **ffmpeg** 包（建议使用 7.1.1，支持 **libsvtav1 编码**）。

```
conda create -n lerobot python=3.10.18 ffmpeg=7.1.1 -c conda-forge
```

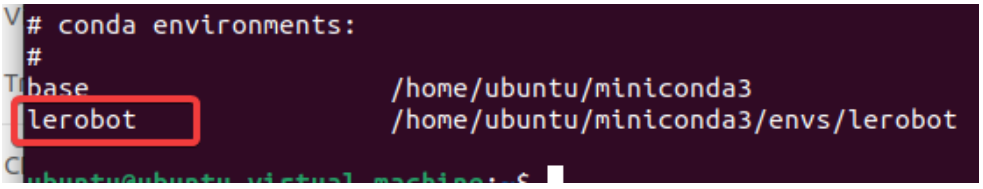
> **Note**
>
> **首次创建环境通常需要约 5～15 分钟。** 如果长时间停留在依赖解析阶段，请耐心等待数分钟；网络较慢或需要重新下载软件包时，整体可能需要 20 分钟左右。

出现下图提示后，按 **y**，再按 **Enter** 继续。



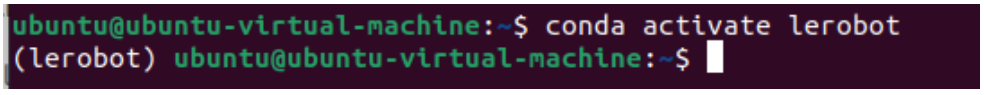
创建完成后，输入指令即可查看已创建的虚拟环境。

```
conda env list
```



②进入虚拟环境

```
conda activate lerobot
```

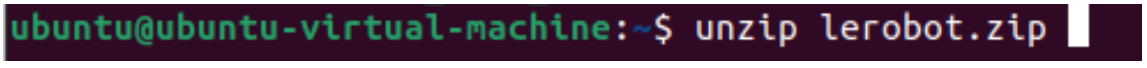


③下载代码库

SO-ARM101 开源 6 轴机械臂工程包路径：**软件工具&源码程序\源码程序\lerobot.zip**

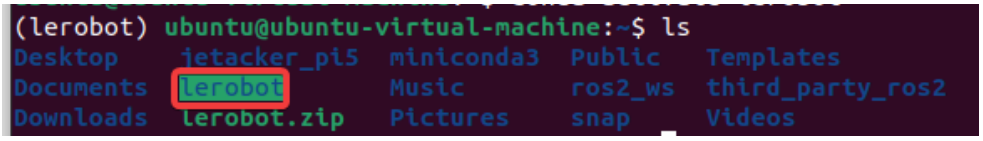
将 **SO-ARM101 开源 6 轴机械臂** 工程包复制到 **Ubuntu** 系统的 `/home/ubuntu` 目录下，然后使用 `unzip` 工具解压。

```
unzip lerobot.zip
```



在终端中输入指令查看文件。

```
ls
```

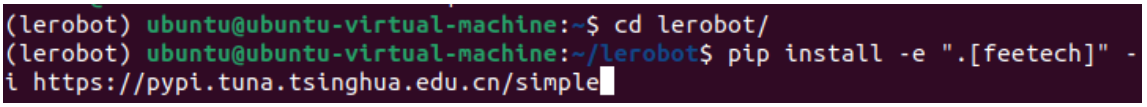


④安装依赖包

进入 **lerobot** 目录，安装 **lerobot** 依赖包，并指定添加舵机相关驱动。

```
cd lerobot
pip install -e ".[feetech]" -i https://pypi.tuna.tsinghua.edu.cn/simple
```

首次安装项目依赖通常需要约 3～10 分钟。等待时间主要取决于网络速度和本机是否已有相关 Python 缓存。



::::

3. 机械臂组装

后续步骤在 **Ubuntu** 和 **Windows** 上基本一致，本节以 **Windows** 为例进行说明。

组装与接线建议按“机械结构 → 摄像头 → 舵机线 → USB 数据线 → 电源”的顺序进行。调整舵机线、拆装支架或确认端口时，应先断开电源，避免带电插拔造成舵机异常或端口识别错误。

> **Note**
>
> **在 Ubuntu 系统上，可能需要运行以下命令，为 USB 端口授予访问权限：**
>
> **sudo chmod 666 /dev/ttyACM0**
> **sudo chmod 666 /dev/ttyACM1**

3.1 硬件组装

查看相关视频，路径：1.课程资料\视频教程\3.2硬件组装

3.2 相机安装

查看相关视频，路径：1.课程资料\视频教程\3.3 相机安装教程

3.3 电路连接

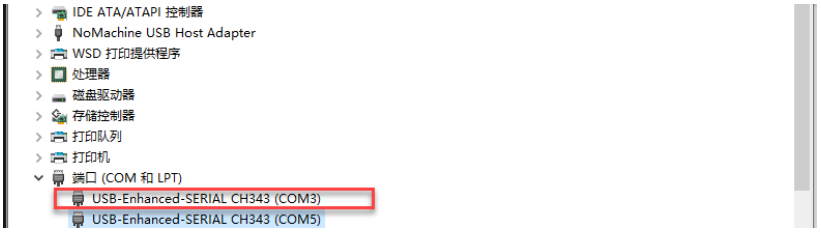
查看相关视频，路径：1.课程资料\视频教程\3.4 电路连接教程

3.4 查找端口号

> **Note**
>
> **文档中的 `COM22` 和 `COM24` 为示例端口号，实际端口号以设备管理器显示为准。如果实际端口不同，后续命令中的端口参数需要同步替换，并注意不要将主臂和从臂端口写反。**

1. 先插入从臂，打开设备管理器，查看端口号。

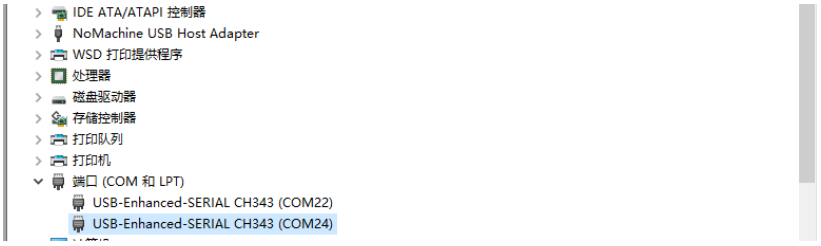
2. 再插入主臂，查看新增的端口号。



3. 为了便于后续使用，可以设置固定端口号。右键 COM 图标，点击属性，进入端口设置 -> 高级。

4. 点击端口号，将主臂端口号设置为 COM22，从臂端口号设置为 COM24。如果 COM22 显示“使用中”，说明该端口已被占用或曾被配置过。

5. 将主臂和从臂端口号分别设置为 22 和 24。设置完成后，如下图所示即为成功。



3.5 舵机 ID 设置（选看）

> **Note**
>
> **本小节适用于 DIY 散件用户，成品用户无需设置 ID，只需了解即可。**

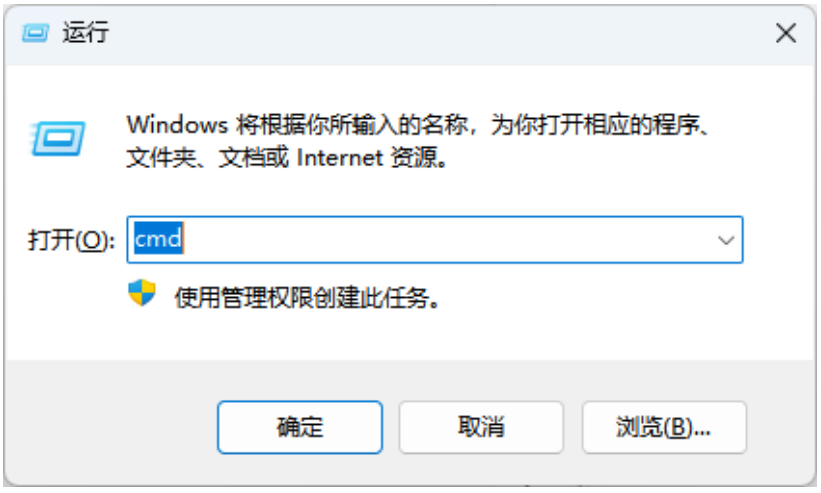
机械臂从上往下各舵机的命名依次为：`gripper`、`wrist_roll`、`wrist_flex`、`elbow_flex`、`shoulder_lift` 和 `shoulder_pan`，分别对应 `ID6` 到 `ID1`。

> **Note**

> **设置舵机 ID 时，只能保留一个舵机连接主控板，其它串联的信号线都需要拔掉；否则可能多个舵机被写成同一个 ID，后续校准和控制会异常。**

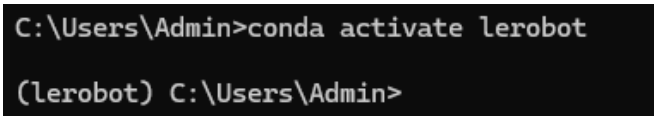
3.5.1 主臂 ID 设置

按 **Win+R**，输入 **cmd**，打开终端。



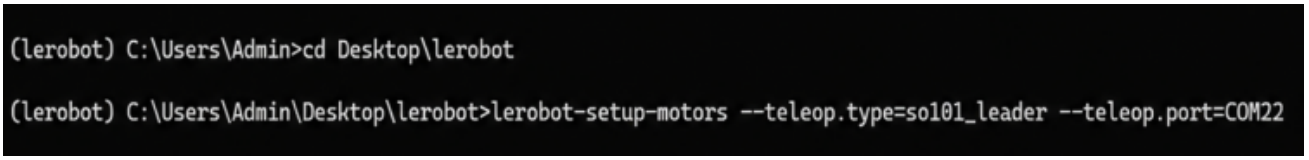
输入指令，进入虚拟环境。

```
conda activate lerobot
```



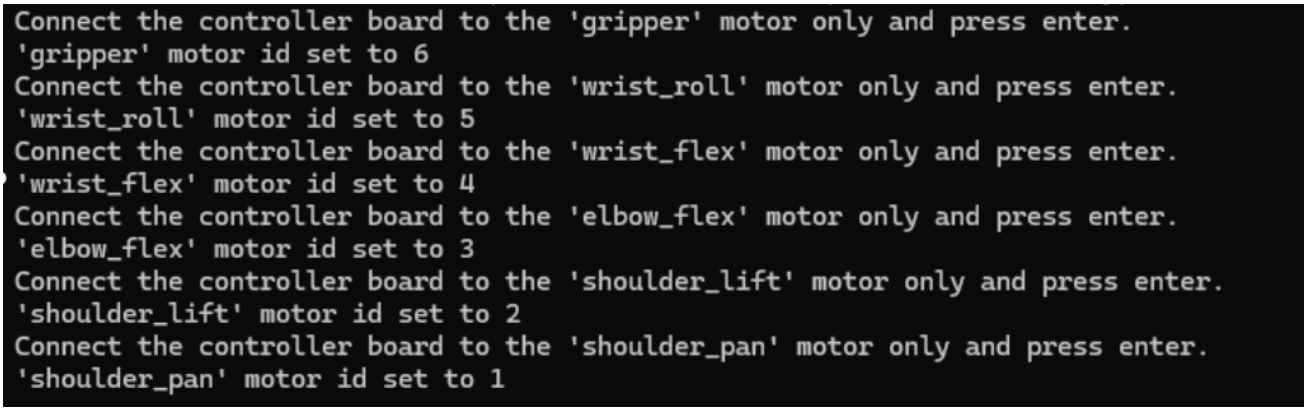
使用 `cd` 工具进入工程文件夹，然后输入主臂指令设置 `ID`。

```
cd Desktop\lerobot
lerobot-setup-motors --teleop.type=so101_leader --teleop.port=COM22
```



每出现一个舵机名提示，都要确保只接入对应 `ID` 的舵机，然后按下回车即可自动设置。当出现下一个舵机名称时，表示上一个舵机已设置完成，后续按提示插拔舵机线并重复回车操作即可。

> **Note**
>
> **注意标记好设置的 ID 号，避免装配时混淆；每个舵机需要安装到对应关节上。**



3.5.2 从臂 ID 设置

从臂与主臂的 ID 设置步骤一致，进入工程文件夹后输入从臂的 ID 设置指令即可，步骤可参考 主臂 ID 设置。

```
lerobot-setup-motors --robot.type=so101_follower --robot.port=COM24
```

```
(lerobot) C:\Users\Admin\Desktop\lerobot>lerobot-setup-motors --robot.type=so101_follower --robot.port=COM24
```

4. 机械臂控制

机械臂控制建议按“校准 → 无视觉遥操作 → 带视觉遥操作”的顺序进行。校准用于建立主从臂关节位置的对应关系；先通过无视觉遥操作确认运动方向和端口正确，再接入摄像头，可避免在视觉采集时同时排查多个问题。

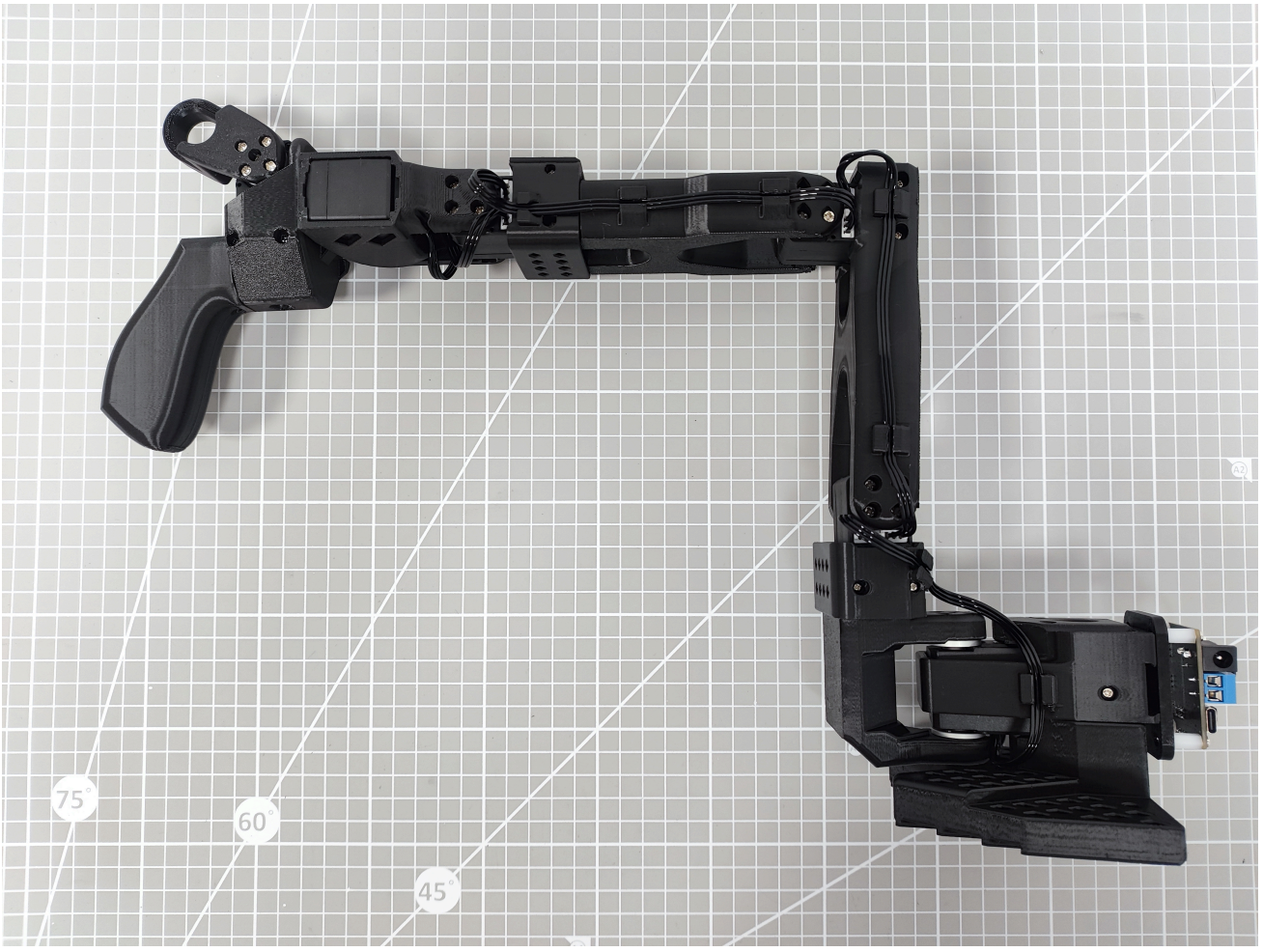
4.1 机械臂校准

> **Note**
>
> **校准和遥操作前，请确保机械臂周围没有遮挡物。若运行过程中出现异常，可以在终端中按 `Ctrl+C` 退出程序。**

接下来需要对机械臂进行校准，确保主臂和从臂处于相同物理位置时，关节位置值保持一致。

4.1.1 主臂校准

校准前，需要将主臂所有关节转动到校准初始位置，姿态如下图所示。



输入校准指令开始校准。

```
lerobot-calibrate --teleop.type=so101_leader --teleop.port=COM22 --teleop.id=my_awesome_leader_arm
```

```
(lerobot) C:\Users\Admin\Desktop\lerobot>lerobot-calibrate --teleop.type=so101_leader
--teleop.port=COM22 --teleop.id=my_awesome_leader_arm
```

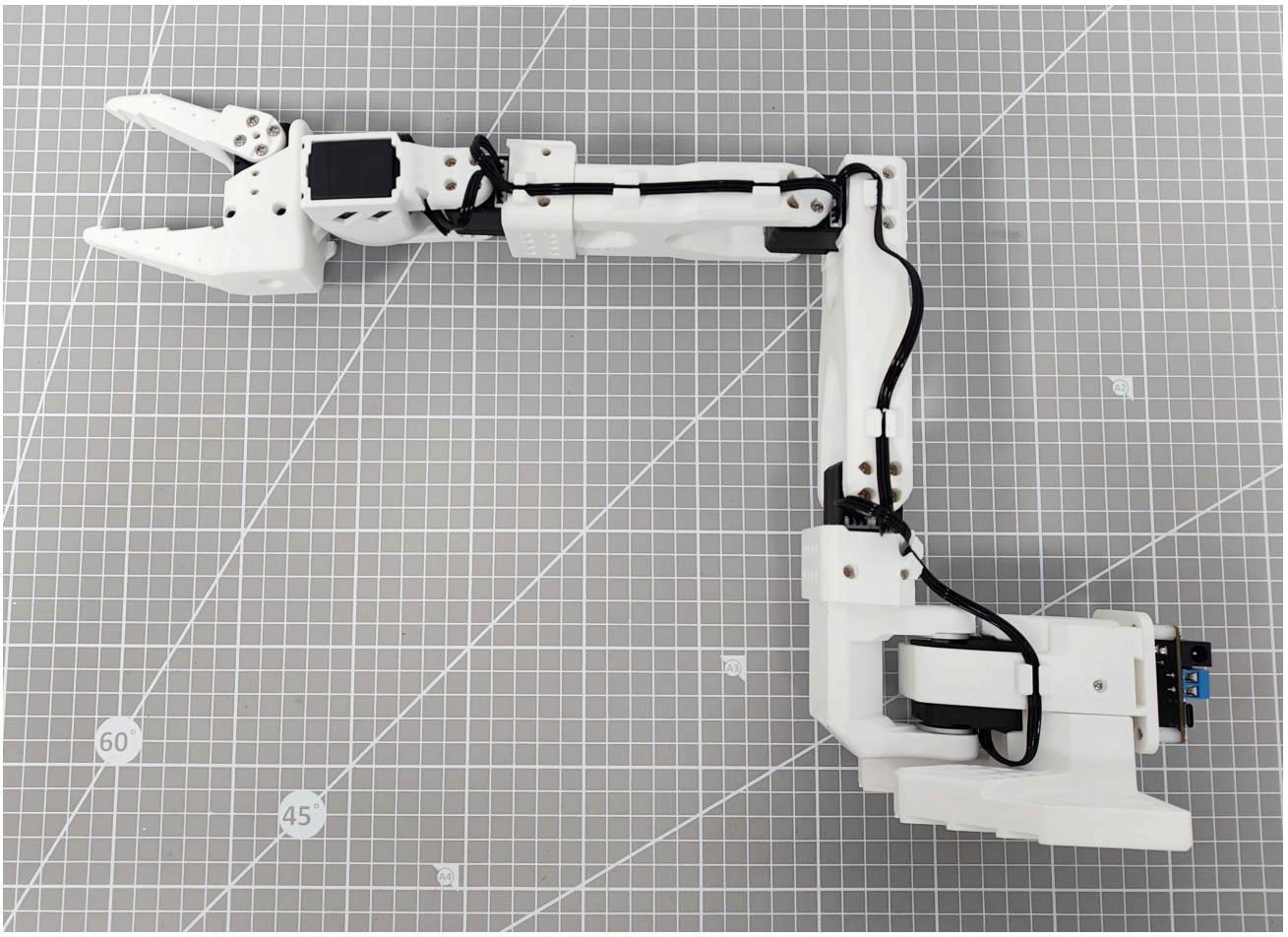
如果之前没有校准过，按下 **Enter** 即可进入校准；如果已经校准过但需要重新校准，需先输入 **c**，再按下 **Enter**。

```
Press ENTER to use provided calibration file associated with the id my_awesome_follower_arm, or type '
c' and press ENTER to run calibration: c
```

校准时，需要手动转动机械臂的每个关节；具体操作可结合从臂校准视频查看。

4.1.2 从臂校准

校准前，需要将从臂所有关节转动到校准初始位置，姿态如下图所示。



从臂与主臂的校准方式一致。进入工程文件夹后输入从臂校准指令即可，具体步骤可参考 主臂校准。

```
lerobot-calibrate --robot.type=so101_follower --robot.port=COM24 --robot.id=my_awesome_follower_arm
```

```
(lerobot) C:\Users\Admin\Desktop\lerobot>lerobot-calibrate --robot.type=so101_follower
--robot.port=COM24 --robot.id=my_awesome_follower_arm
```

4.2 遥操作不带视觉

输入指令，启动机械臂遥操作。

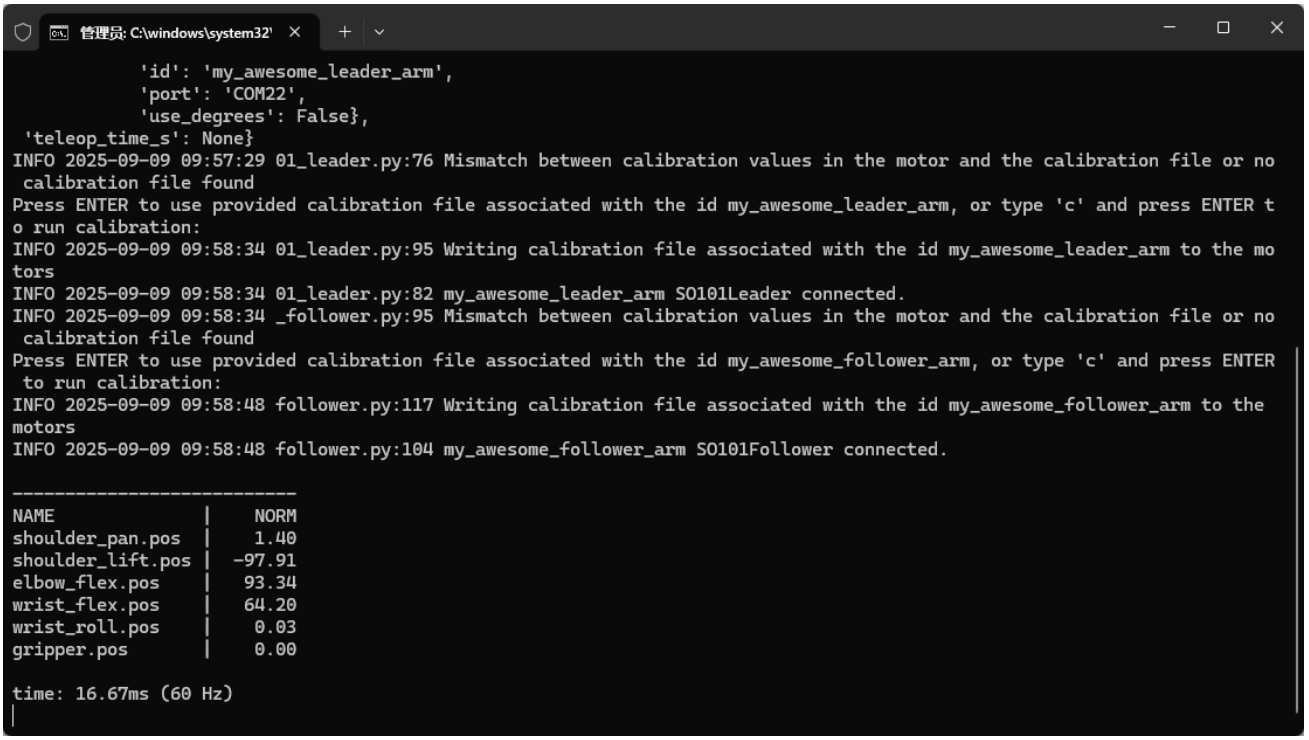
```
python -m lerobot.teleoperate --robot.type=so101_follower --robot.port=COM24 --
robot.id=my_awesome_follower_arm --teleop.type=so101_leader --teleop.port=COM22 --
teleop.id=my_awesome_leader_arm
```

```
(lerobot) C:\Users\Admin\Desktop\lerobot>python -m lerobot.teleoperate --robot.type=so101_follower --robot.port=COM24 --
robot.id=my_awesome_follower_arm --teleop.type=so101_leader --teleop.port=COM22 --teleop.id=my_awesome_leader_arm
```

出现下图提示时，按下 **Enter** 继续。

```
Press ENTER to use provided calibration file associated with the id my_awesome_leader_arm, or type 'c' and press ENTER t
o run calibration:
```

出现下图界面后，即可通过主臂遥控从臂。



按下 **Ctrl+c** 即可终止程序。

4.3 遥操作带视觉

将两个摄像头的 USB 接入电脑。

> **Note**
>
> **①如果使用拓展坞，两个摄像头的 USB 不能都接在拓展坞上。**
>
> **②固定环境摄像头的画面必须能够看到整个从臂的动作。**

输入指令，查找摄像头 `ID`，并检查是否能获取到两个摄像头的画面。

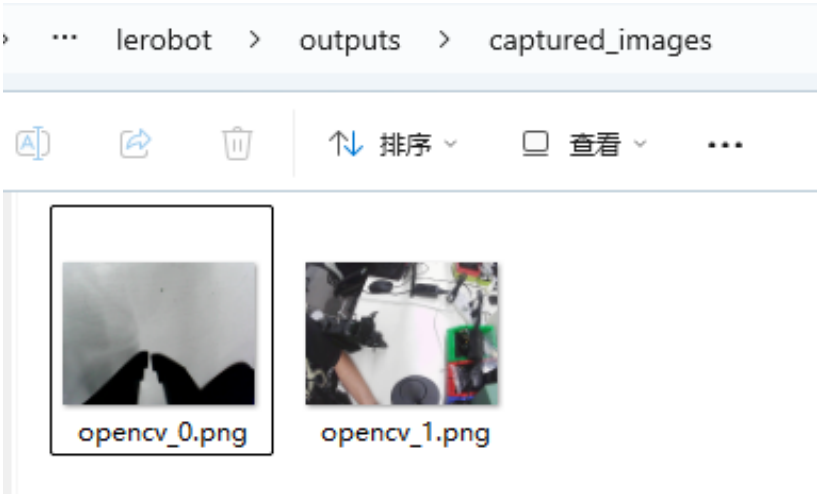
```
lerobot-find-cameras opencv
```

摄像头检测通常需要约 10～30 秒，程序会依次打开可识别的摄像头并保存测试画面。若摄像头数量较多、首次连接 USB 设备或电脑性能较低，等待时间可能延长至 1 分钟左右。

```
--- Detected Cameras ---
Camera #0:
  Name: OpenCV Camera @ 0
  Type: OpenCV
  Id: 0
  Backend api: MSMF
  Default stream profile:
    Format: -1.0
    Width: 640
    Height: 480
    Fps: 30.0
--------------------
Camera #1:
  Name: OpenCV Camera @ 1
  Type: OpenCV
  Id: 1
  Backend api: MSMF
  Default stream profile:
    Format: -1.0
    Width: 640
    Height: 480
    Fps: 30.0
--------------------
```

运行结束后，捕获的图片会保存到 `outputs\captured_images`。

```
Image capture finished. Images saved to outputs\captured_images
```



可以通过文件名来区分摄像头 `ID`：`opencv_0` 对应 `index_or_path: 0`，`opencv_1` 对应 `index_or_path: 1`。

> **Note**
>
> **摄像头 `ID` 不是永久固定的。拔插 USB、更换 USB 接口或更换电脑后，`opencv_0` 和 `opencv_1` 的顺序可能变化，运行带视觉遥操作或数据采集前建议重新确认摄像头画面。**

运行指令，启动带视觉画面的遥操作。

```
python -m lerobot.teleoperate --robot.type=so101_follower --robot.port=COM24 --
robot.id=my_awesome_follower_arm --robot.cameras="{ fixed: {type: opencv, index_or_path: 1, width: 640,
height: 480, fps: 30}, handeye: {type: opencv, index_or_path: 0, width: 640, height: 480, fps: 30}}" --
teleop.type=so101_leader --teleop.port=COM22 --teleop.id=my_awesome_leader_arm --display_data=true
```

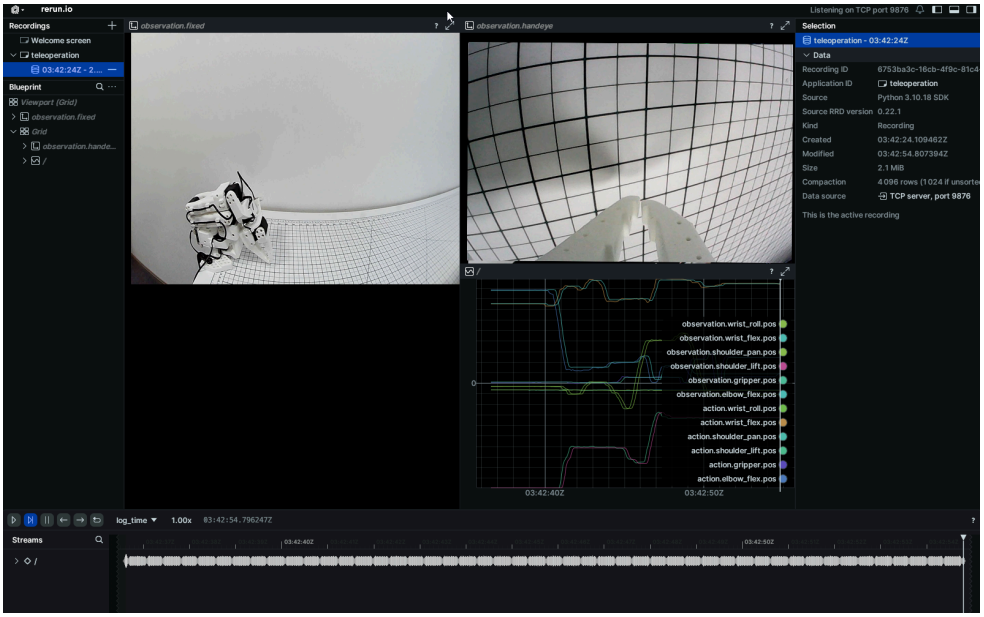
```
(lerobot) C:\Users\Admin\Desktop\lerobot>python -m lerobot.teleoperate --robot.type=so101_follower --robot.port=COM24 --
robot.id=my_awesome_follower_arm --robot.cameras="{ fixed: {type: opencv, index_or_path: 1, width: 640, height: 480, fps
: 30}, handeye: {type: opencv, index_or_path: 0, width: 640, height: 480, fps: 30}}" --teleop.type=so101_leader --teleop
.port=COM22 --teleop.id=my_awesome_leader_arm --display_data=true
```

> **Note**
>
> `fixed` **为固定环境摄像头，** `handeye` **为夹爪相机。**

出现下图提示时，按下 **Enter** 继续。

```
Press ENTER to use provided calibration file associated with the id my_awesome_follower_arm, or type 'c' and press ENTER
 to run calibration:
```

通过主臂控制从臂时，系统会同步回传摄像头画面。若想关闭程序，在终端中按 **Ctrl+c** 即可。



5. 数据采集

完成遥操作验证后，即可开始采集训练数据。数据质量会直接影响后续模型训练效果，建议每轮录制前先确认摄像头视野、光照、目标物体位置和机械臂初始姿态。

数据采集步骤：

1. 初始化机器人和传感器。
2. 启动遥操作界面。
3. 操作者控制机器人完成任务。
4. 系统同步记录所有传感器数据和控制指令。
5. 数据被保存为标准 LeRobot 数据集格式。
6. 可选的实时质量监控和可视化。

> **Note**
>
> **数据采集耗时参考：** 启动采集程序并初始化机械臂、摄像头通常需要 20～60 秒。每一轮数据的总耗时由实际示教动作、环境复位和数据保存组成；保存阶段一般需要数秒到几十秒。以 20 轮数据为例，建议预留至少 30～90 分钟，并在开始前准备好足够的桌面空间和稳定光照。

运行指令进行数据采集。`${HF_USER}` 可以改为自己的用户名（注意用户名必须为英文）。这里以采集 20 次为例，如果需要采集更多轮次，可将 `dataset.num_episodes=20` 中的 `20` 改为目标次数。

> **Note**
>
> **1、数据采集前，需要先熟练控制机械臂完成夹取动作，避免因操作不规范导致数据不准确。**
>
> **2、指令中可以直接把命令中的** `${HF_USER}` **替换成自己的英文用户名，例如将** `--dataset.repo_id=${HF_USER}/demo` **改成** `--dataset.repo_id=Admin/demo`。

```
python -m lerobot.record --robot.type=so101_follower --robot.port=COM24 --robot.id=my_awesome_follower_arm --robot.cameras="{ handeye: {type: opencv, index_or_path: 0, width: 640, height: 480, fps: 30}, front: {type: opencv, index_or_path: 1, width: 640, height: 480, fps: 30}}" --teleop.type=so101_leader --teleop.port=COM22 --teleop.id=my_awesome_leader_arm --display_data=true --dataset.repo_id=${HF_USER}/demo --dataset.num_episodes=20 --dataset.single_task="Grab the screwdriver" --dataset.push_to_hub=false
```

命令启动后，机器人连接与摄像头初始化通常需要约 20～60 秒。每轮录制结束后的数据编码和保存通常需要数秒到几十秒，摄像头数量更多、分辨率更高时等待时间会增加；请在保存进度完成前不要关闭终端或拔出 USB 设备。

```
(lerobot) C:\Users\Admin\Desktop\lerobot>python -m lerobot.record --robot.type=so101_follower --robot.port=COM24 --robot
.id=my_awesome_follower_arm --robot.cameras="{ handeye: {type: opencv, index_or_path: 0, width: 640, height: 480, fps: 3
0}, front: {type: opencv, index_or_path: 1, width: 640, height: 480, fps: 30}}" --teleop.type=so101_leader --teleop.port
=COM22 --teleop.id=my_awesome_leader_arm --display_data=true --dataset.repo_id=Admin/demo --dataset.num_episodes=20 --da
taset.single_task="Grab the screwdriver"
```

若出现下图提示，可按下 **Enter** 继续下一步操作。

```
Press ENTER to use provided calibration file associated with the id my_awesome_follower_arm, or type 'c' and press ENTER
 to run calibration:
```

出现下图提示时，说明已经进入数据录制。此时可以操控主臂，让从臂夹取目标物体并放置到目标位置。

```
INFO 2025-09-09 15:14:03 01_leader.py:82 my_awesome_leader_arm SO101Leader connected.
INFO 2025-09-09 15:14:03 ls\utils.py:227 Recording episode 0
```

> **Note**
>
> **录制时，除机械臂外，环境最好保持相对静止。例如摄像头画面中尽量只出现机械臂运动，手臂、人员等无关内容不要进入画面。**

动作执行完毕后，按下 → 继续。出现下图提示后，即可将环境复位。

```
INFO 2025-09-09 15:22:14 ls\utils.py:227 Reset the environment
```

环境复位完毕后，按下 → 保存数据。当下图所示进度条加载到 100% 时，说明数据保存完毕。如果不想保留当前轮数据，按 ← 即可重新录制，本轮数据不会保存。

```
Map: 100%|                                                                    | 1199/1199 [00:00<00:00, 2810.32 examples/s]
Creating parquet from Arrow format: 100%|                                     | 2/2 [00:00<00:00, 1963.17ba/s]
```

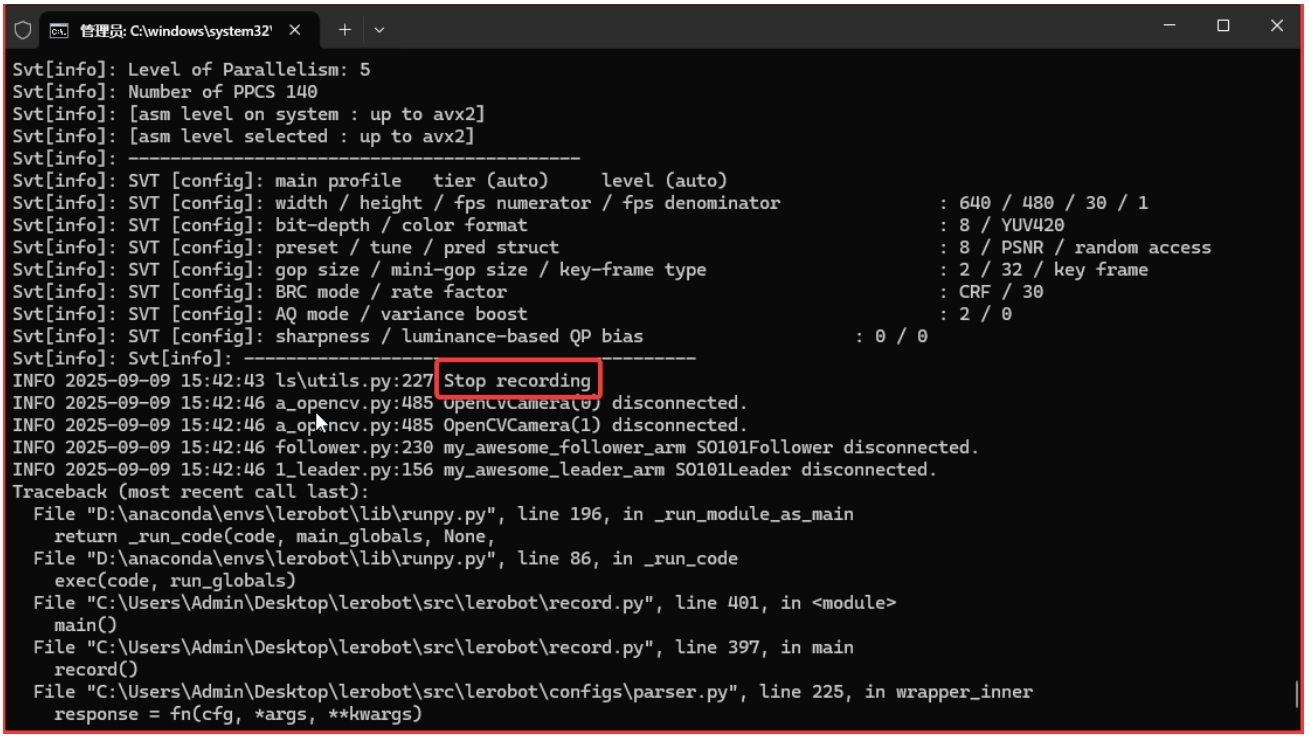
上一轮录制结束后，系统会等待下一次录制。出现下图标志时，说明已经进入下一轮数据录制。

```
INFO 2025-09-09 15:34:00 ls\utils.py:227 Recording episode 1
```

这里以采集 20 轮数据为例。为了提高数据多样性，每轮目标物体的放置位置最好不要完全相同。

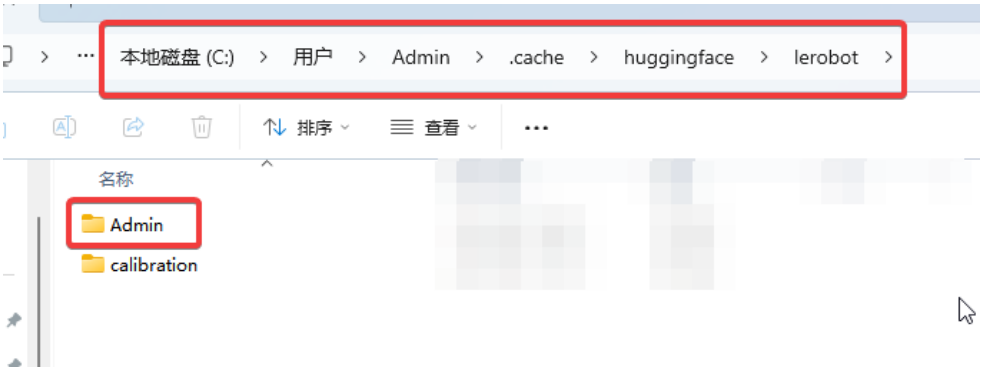
```
INFO 2025-09-09 15:42:42 ls\utils.py:227 Recording episode 19
Right arrow key pressed. Exiting loop...
Map: 100%|                                                                    | 9/9 [00:00<00:00, 1286.07 examples/s]
Creating parquet from Arrow format: 100%|                                     | 1/1 [00:00<?, ?ba/s]
```

当显示 **Recording episode 19** 时，说明已进入第 20 轮录制。录制完成后会打印 **Stop recording**，表示数据采集完成，按下 **Esc** 终止程序即可。



数据默认保存到 `C:\Users\Admin\.cache\huggingface\lerobot` 中。

> **Note**
>
> **实际缓存路径会随 Windows 用户名和 `dataset.repo_id` 改变。如果电脑用户名不是 `Admin`，需要到自己用户目录下的 `.cache\huggingface\lerobot` 中查找数据集。**



6. 数据集训练

数据集训练可以选择云服务器或本机两种方式。云服务器适合没有高性能显卡的电脑，本机训练适合已具备独立显卡和 CUDA 环境的电脑。无论选择哪种方式，都需要先确认数据集已采集完成，并且能在默认缓存目录中找到对应数据。

6.1 云服务器训练

6.1.1 云主机租用

本课程选用的云主机平台为智星云。

[智星云 AI Galaxy | GPU云服务器 GPU服务器租用 远程GPU租用 深度学习服务器 | 免费GPU 便宜GPU]

如下图所示，先在右上角登录；如果没有账号，需要先注册。

在算力市场中根据需求选择对应的 **GPU 云主机**。本课程采用的主机为 **RTX4090**，训练时长约两个小时，配置如下。

> **Note**
>
> **RTX4090 1GPU 8核16G 系统200G**
>
> **镜像 22.04 带 cuda12.4，带宽建议选择 80 左右**

选择好系统镜像后，点击 **立即租用**。

根据自己的需求选择计费方式。此次数据集训练大概需要 3 个小时，建议租用结束后保留全部磁盘，防止数据丢失。勾选好后，点击 **创建实例**。

点击 **控制台**，即可看到刚创建的实例。

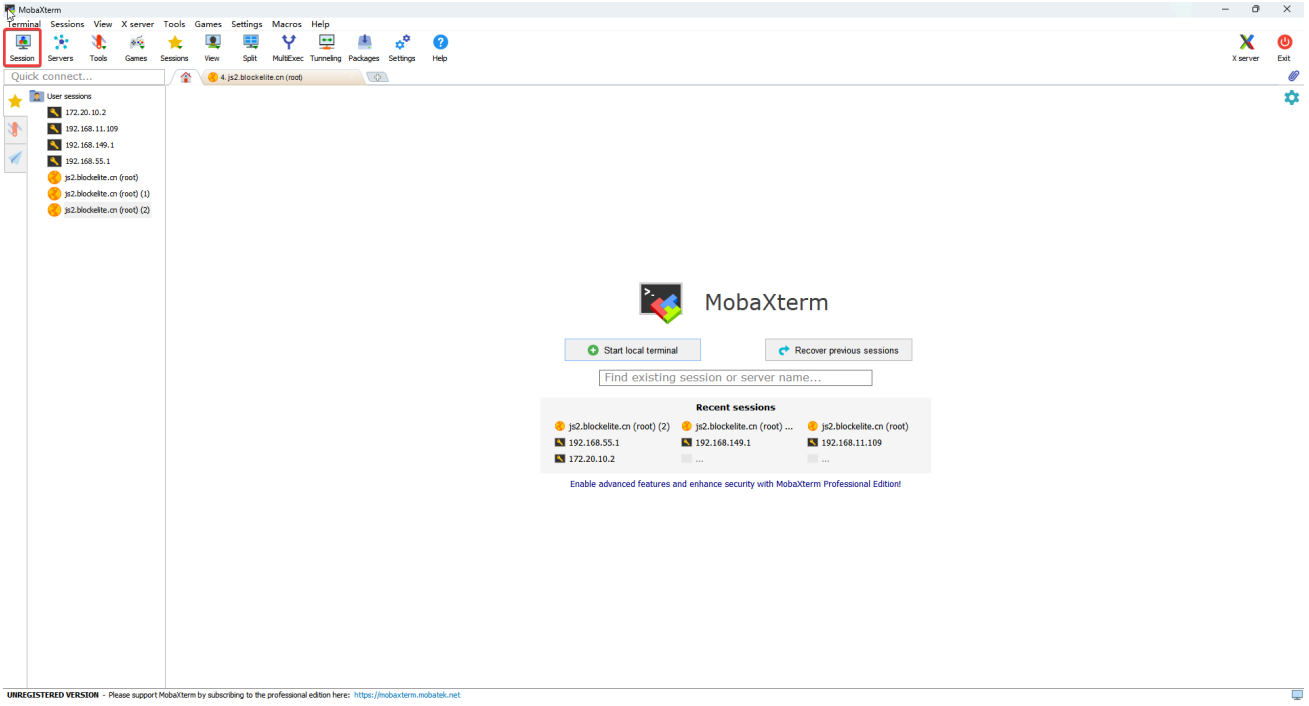
6.1.2 连接方式查看

点击查看 **连接方式**，选择 SSH 连接，即可看到 **地址、账户、端口和密码**。

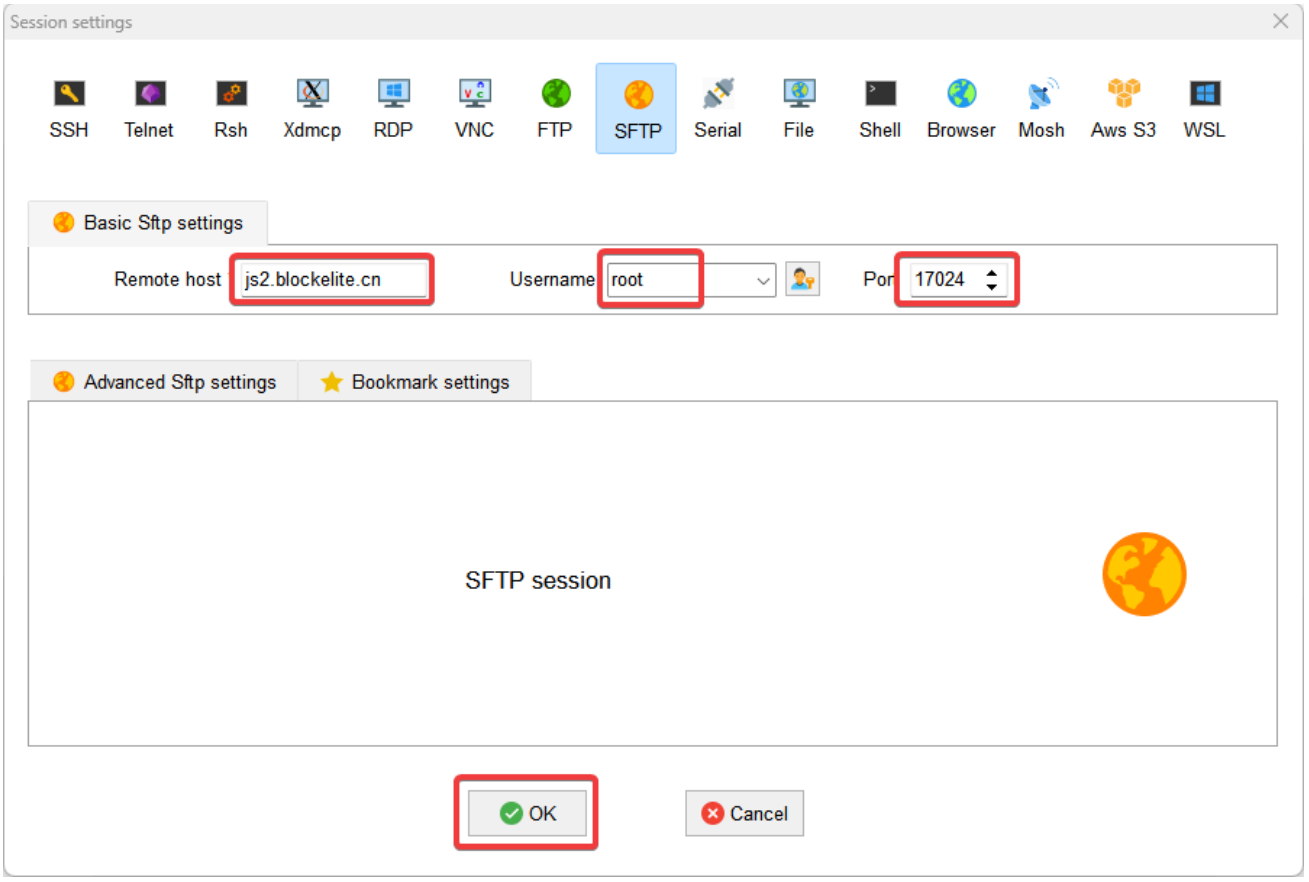
6.1.3 数据集上传

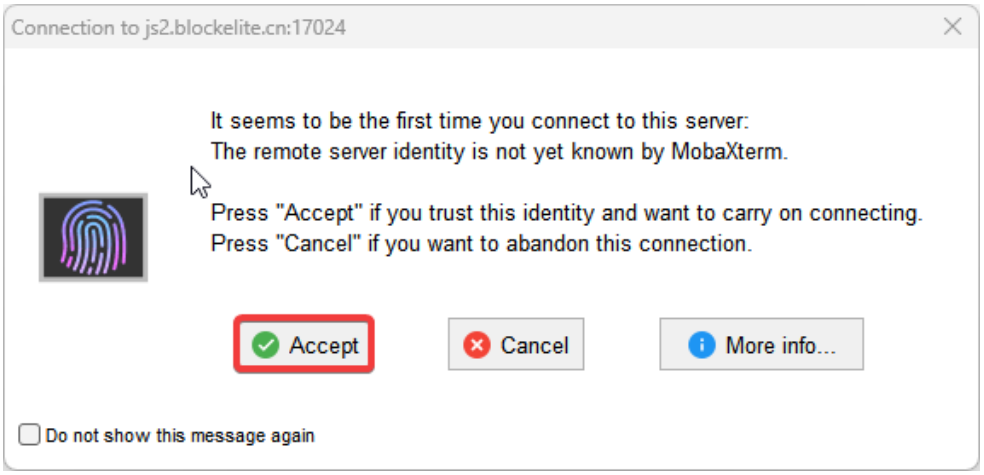
使用 `MobaXterm` 进行文件传输。打开 `MobaXterm` 后，点击 **Session**`<img src="../_static/media/chapter_1/section_6/image_7.png" class="inline-icon" style="width:30px;"/>`。

`MobaXterm` 远程连接软件下载路径：`软件工具&源码程序\软件工具\MobaXterm_Installer_v22.1.zip`

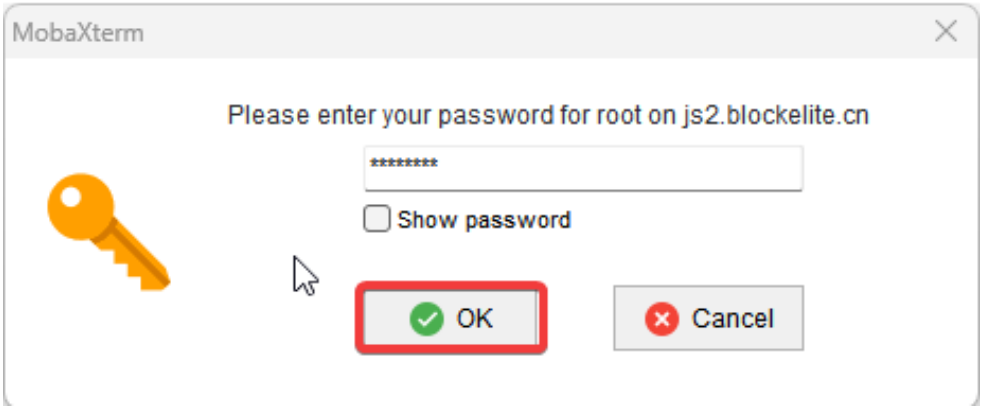


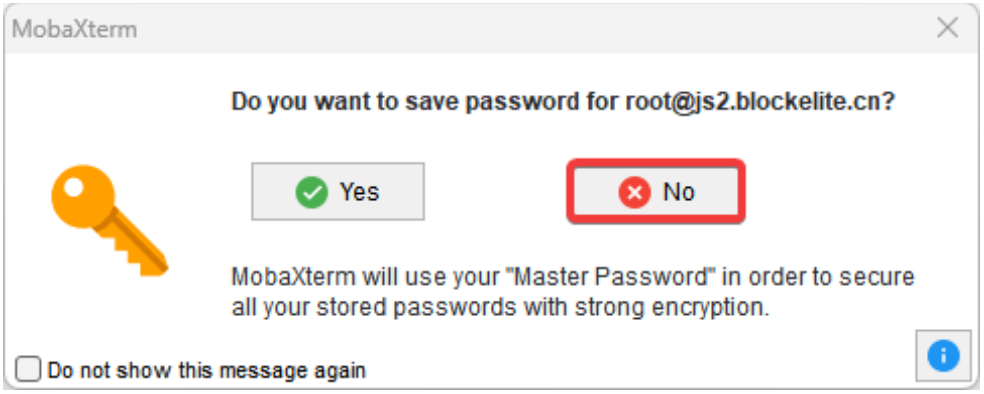
使用 **SFTP** 进行文件传输，将云主机的 **地址、用户名和端口** 填写进去，如下图所示，然后点击 **OK**。


Session settings
SSH
Telnet
Rsh
Xdmcp
RDP
VNC
FTP
SFTP
Serial
File
Shell
Browser
Mosh
Aws S3
WSL
Basic Sftp settings
Remote host
js2.blockelite.cn
Username
root
Port
17024
Advanced Sftp settings
Bookmark settings
SFTP session
OK
Cancel


Connection to js2.blockelite.cn:17024
It seems to be the first time you connect to this server:
The remote server identity is not yet known by MobaXterm.
Press "Accept" if you trust this identity and want to carry on connecting.
Press "Cancel" if you want to abandon this connection.
Accept
Cancel
More info...
Do not show this message again

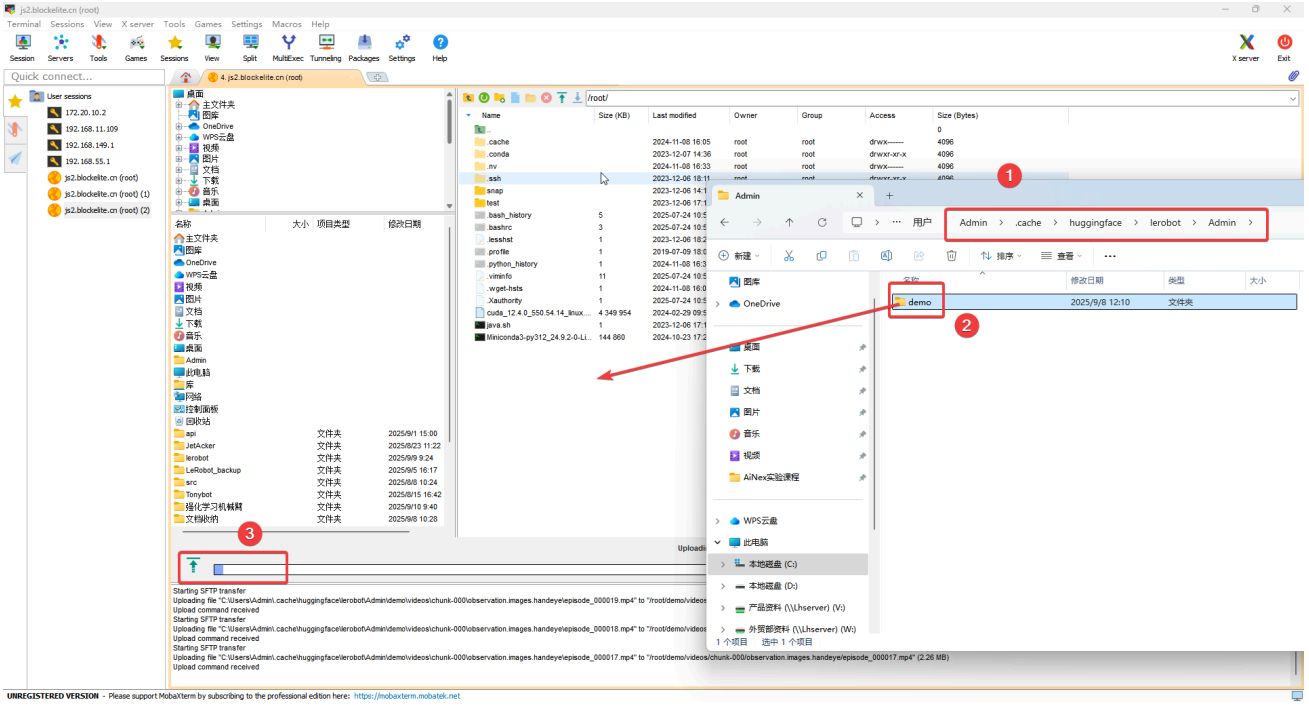
输入云主机密码并登录。


MobaXterm
Please enter your password for root on js2.blockelite.cn
****
Show password
OK
Cancel

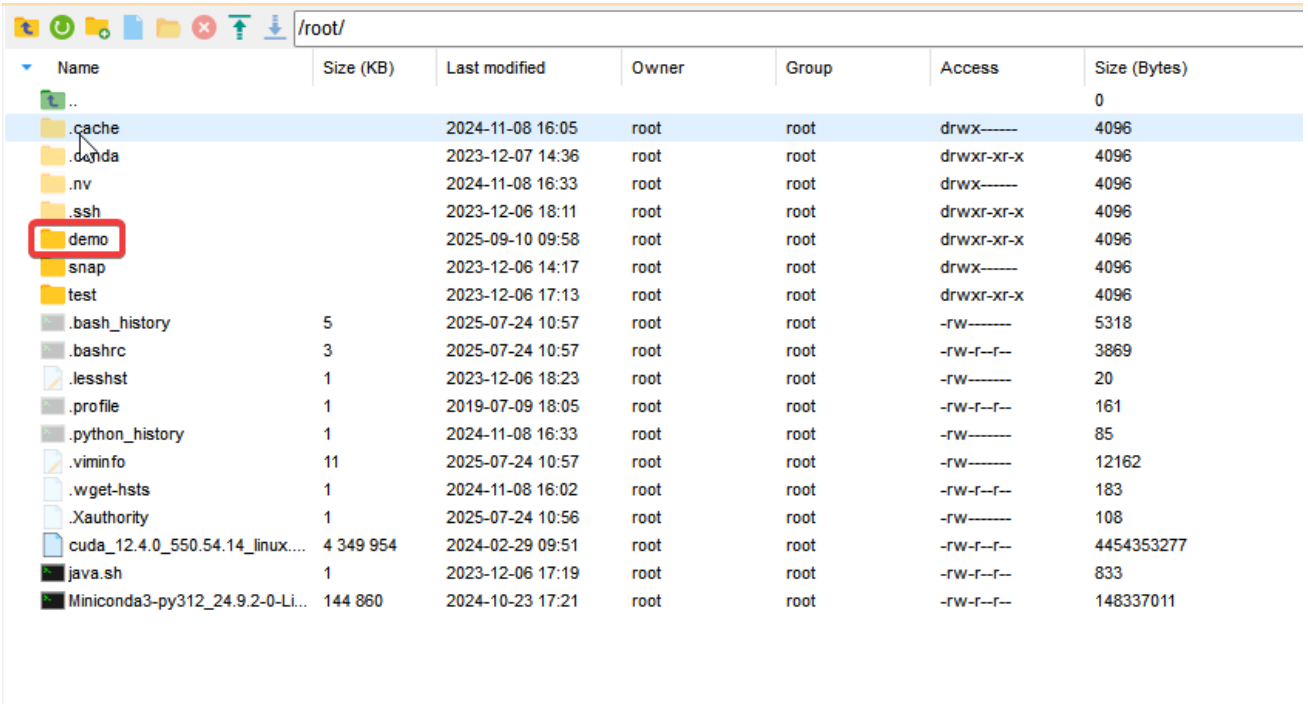


打开文件夹，找到采集的数据。这里示例存放路径为 `C:\Users\Admin\.cache\huggingface\lerobot\Admin`，将 **demo** 文件夹整个拖到云主机上。

数据集上传时间取决于数据集大小和本地上行带宽，可能从数分钟到数十分钟不等。传输过程中请保持 SFTP 连接，不要只上传其中某个子文件夹；云端目录结构需要与本地采集结果保持完整。



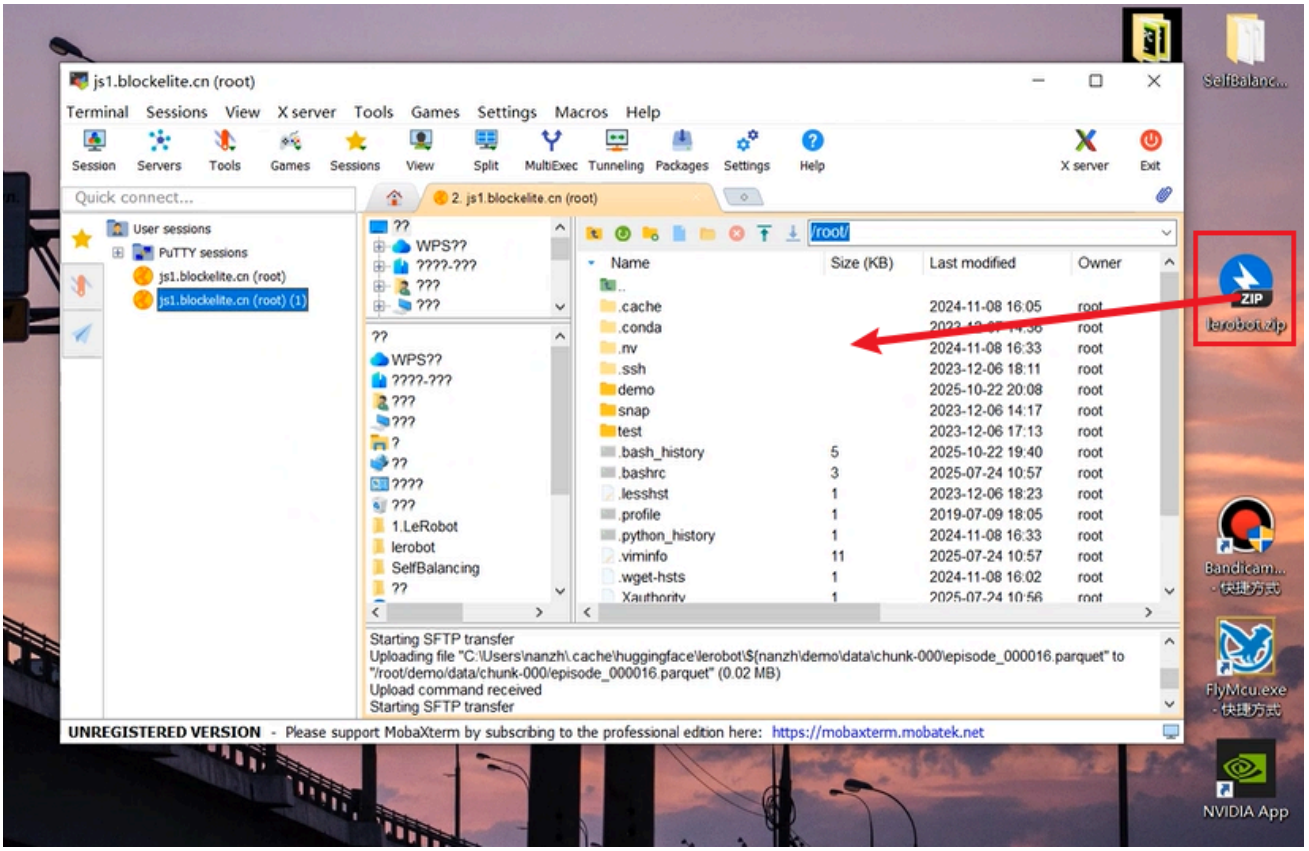
传输完成后，可查看 `root` 目录下是否有一个 **demo** 文件夹。



6.1.4 lerobot 工程包上传

将 **lerobot.zip** 文件拖到云主机上。

工程包通常比数据集小，上传一般需要数秒到数分钟；完成后再继续解压操作。

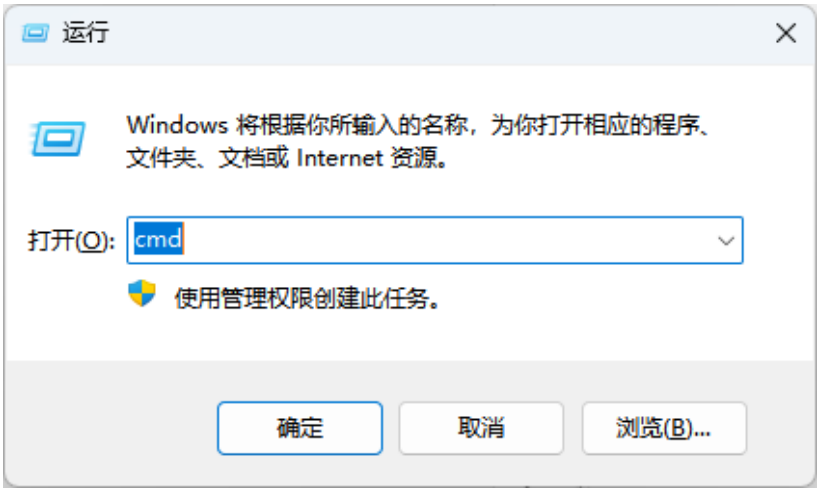


传输完成后，可查看 `root` 目录下是否有一个 **lerobot.zip** 文件。

6.1.5 数据集训练

点击复制 **SSH 登录命令**。

按 **Win+R**，输入 **cmd**，打开终端。



将指令复制到终端中，按下 **Enter**，输入 **yes** 后再按下 **Enter**。

```
C:\Users\Admin>ssh root@js2.blockelite.cn -p 17024
The authenticity of host '[js2.blockelite.cn]:17024 ([180.127.11.167]:17024)' can't be established.
ED25519 key fingerprint is SHA256:WEeKFd90sfhAztcvbci9bH98kReJlkMNgvlkskxTnJA.
This host key is known by the following other names/addresses:
    C:\Users\Admin/.ssh/known_hosts:13: [js2.blockelite.cn]:16928
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
```

输入云主机密码。

出现下图界面，表示登录成功。

使用 `apt` 工具下载 **screen**，这是一个终端复用器，可以在一个终端窗口中创建多个虚拟终端会话。

> **Note**
>
> `screen` 的作用是让训练任务在后台持续运行。云服务器训练通常需要数小时，如果直接在普通 SSH 终端中运行，电脑休眠、网络波动或关闭终端都可能导致连接断开；将任务放入 `screen` 会话后，即使退出 SSH 窗口，服务器上的训练进程仍会继续执行，之后可重新连接会话查看进度。

```
apt install -y screen
```

安装通常只需要几十秒到 2 分钟；如果提示需要确认，请按终端提示继续。

使用 `unzip` 工具解压 **lerobot.zip** 文件。

```
unzip lerobot.zip
```

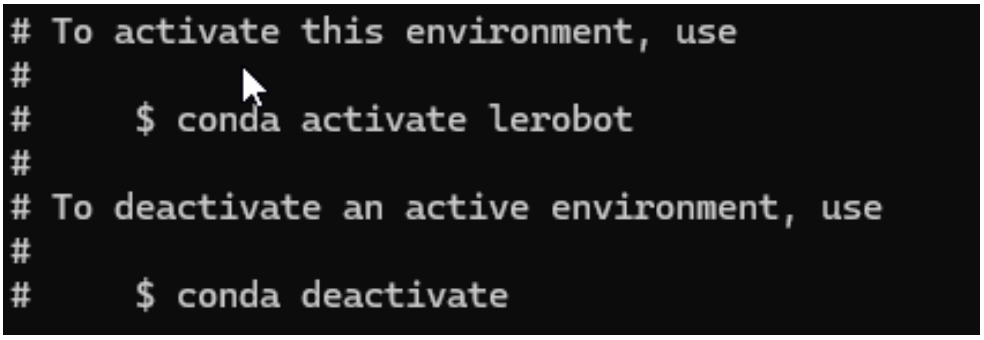
工程包解压通常需要 1～3 分钟，具体取决于云服务器磁盘性能和工程包大小。

使用 **Miniconda** 创建虚拟环境并安装 **ffmpeg**。

```
conda create -n lerobot python=3.10.18 ffmpeg=7.1.1 -c conda-forge
```

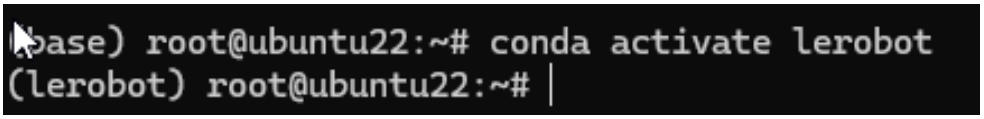
> **Note**
>
> **云服务器首次创建环境通常需要约 5～15 分钟。** 依赖解析、软件包下载和安装都会消耗时间；若镜像站连接较慢或服务器刚创建，等待 20 分钟左右也属正常，请不要中途终止命令。

出现下图所示提示时，说明创建成功。



激活虚拟环境。

```
conda activate lerobot
```



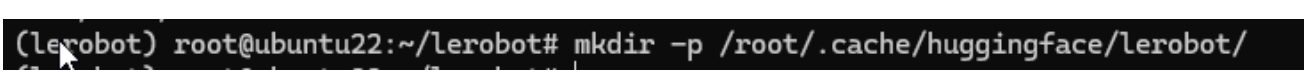
使用 `cd` 工具跳转到 **lerobot** 目录下，安装 **lerobot** 的依赖包，并指定添加舵机相关驱动。

```
cd lerobot
python -m pip install -e ".[feetech,training]" -i https://pypi.tuna.tsinghua.edu.cn/simple
```

训练依赖首次安装通常需要约 5～20 分钟。该过程会下载深度学习训练相关的 Python 包，下载完成后再进行下一步即可。

输入指令，创建数据集存储目录。

```
mkdir -p /root/.cache/huggingface/lerobot/
```



将 `/root/demo` 移到 `/root/.cache/huggingface/lerobot` 下。

```
mv /root/demo/ /root/.cache/huggingface/lerobot/
```

```
(lerobot) root@ubuntu22:~/lerobot# mv /root/demo/ /root/.cache/huggingface/lerobot/
```

创建一个名为 **lerobot** 的 `screen` 会话，完成后将会自动进入`screen`会话内。建议在此会话内执行训练命令，以避免本地 SSH 连接中断影响训练。

```
screen -S lerobot
```

```
(lerobot) root@ubuntu22:~/lerobot# screen -S lerobot
```

> **Note**
>
> - 如需退出 `screen` 会话，按 **Ctrl+a** 之后再单独按一下d键即可退出。
> - 注意：退出后`screen`会话并不会销毁。
> - 退出了`screen`会话之后，想要重新进入会话，输入以下指令即可：`screen -r lerobot`

进入虚拟终端后，需要重新激活虚拟环境。

```
conda activate lerobot
```

```
(base) root@ubuntu22:~# conda activate lerobot
(lerobot) root@ubuntu22:~#
```

使用 `cd` 指令跳转到 `~/lerobot/src` 目录下。

```
(lerobot) root@ubuntu22:~/lerobot# cd ~/lerobot/src/
```

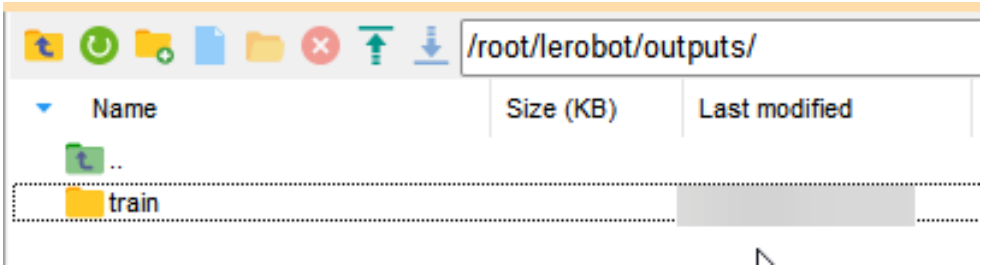
输入指令开始训练。首次启动时会进行数据集读取、模型初始化和 CUDA 环境检查，通常需要等待 1～5 分钟后才会持续输出训练日志。

```
python lerobot/scripts/train.py --dataset.repo_id=demo --policy.type=act --
output_dir=outputs/train/act_so101_test --job_name=act_so101_test --policy.device=cuda --wandb.enable=false  --
policy.push_to_hub=false
```

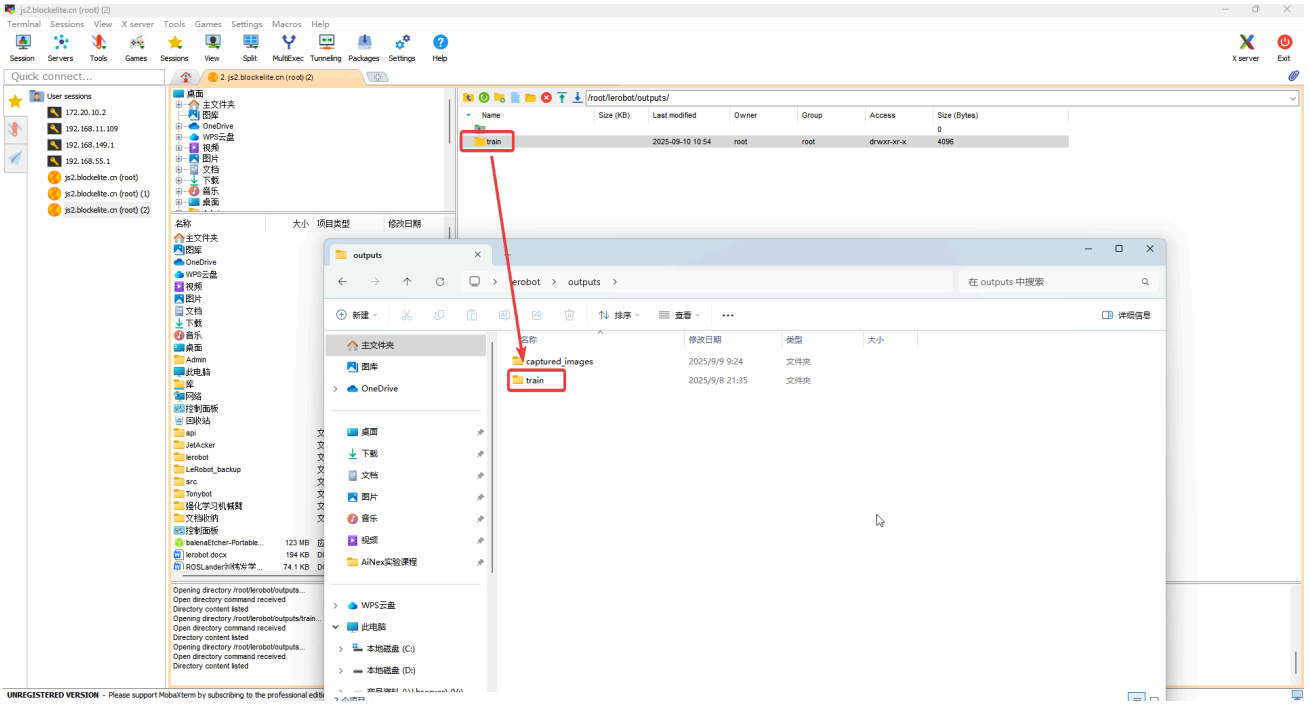
> **Note**
>
> **训练等待时间参考：** 以本教程示例的 RTX4090、默认 `100000` 步训练为例，通常约需 2～3 小时。实际时间会受到 GPU 型号、数据集规模、摄像头数量和服务器负载影响；训练期间可按 **Ctrl+a** 后再按 **d** 退出 `screen` 会话，训练不会停止。

```
(lerobot) root@ubuntu22:~/lerobot/src# python lerobot/scripts/train.py --dataset.repo_id=demo --policy.type=act --output
_dir=outputs/train/act_so101_test --job_name=act_so101_test --policy.device=cuda --wandb.enable=false  --policy.push_to_
hub=false
```

训练完成后，将在 `lerobot/outputs` 下生成一个 `train` 文件夹，具体路径为 `/root/lerobot/outputs/train`。



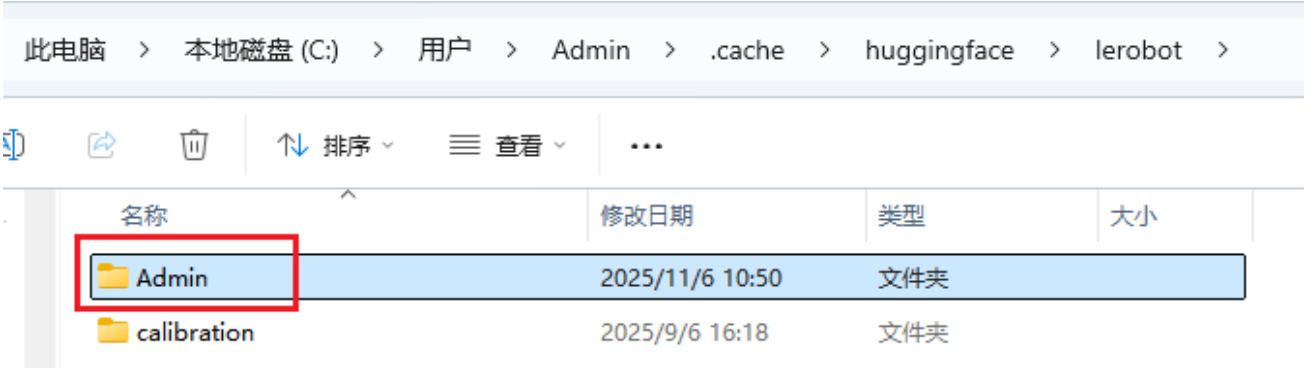
继续使用 **SFTP**，将训练好的文件传输到本机工程目录下，路径为：`lerobot\outputs`。



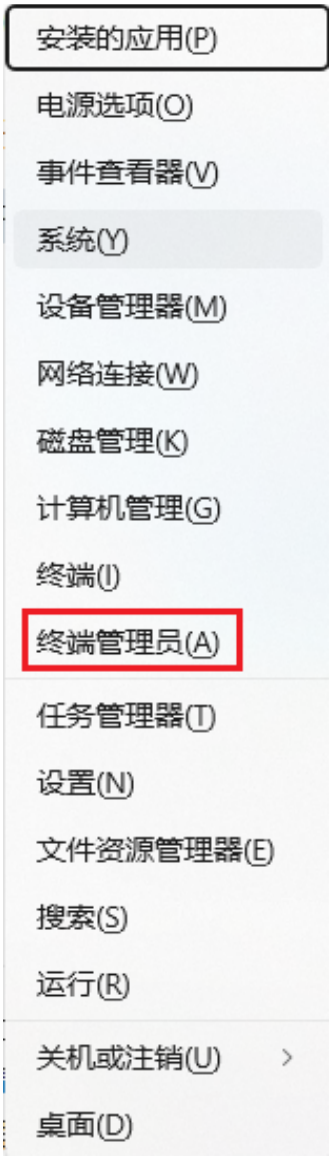
至此，云服务器训练步骤完成，接下来即可进行推理测试。

6.2 本机训练

数据集默认保存到 `C:\Users\Admin\.cache\huggingface\lerobot` 中。



按 **Win+X**，点击 `终端管理员`，以管理员权限打开命令行终端。



输入指令，激活虚拟环境并进入 **lerobot** 工作目录。

```
conda activate lerobot
cd .\Desktop\lerobot\
```

```
(base) PS C:\Users\Admin> conda activate lerobot
(lerobot) PS C:\Users\Admin> cd .\Desktop\lerobot\
```

在训练前，还需要输入指令安装用于训练的依赖包。

```
python -m pip install -e ".[training]" -i https://pypi.tuna.tsinghua.edu.cn/simple
```

首次安装训练依赖通常需要约 5～20 分钟。安装过程中会下载多个 Python 包，网络较慢时请耐心等待；重复安装时会优先使用本地缓存，所需时间通常更短。

> **Note**
>
> - **不推荐在没有独立显卡的电脑上进行本机训练。**
> - **下面的步骤需要电脑配备独立显卡后方可操作。**
> - **安装 PyTorch 前，建议先在终端中输入 `nvidia-smi`，确认电脑可以正常识别 NVIDIA 显卡和驱动。**

输入指令，安装 **CUDA 11.8** 对应的 PyTorch 版本。

```
pip install torch==2.7.1 torchvision==0.22.1 torchaudio==2.7.1 --index-url
https://download.pytorch.org/whl/cu118
```

PyTorch 安装包体积较大，首次下载和安装通常需要约 5～20 分钟；网络较慢时可能更久。安装完成后可执行 `nvidia-smi` 或在 Python 中导入 `torch` 检查显卡环境是否正常。

```
(lerobot) PS C:\Users\Admin\Desktop\lerobot> pip install torch==2.7.1 torchvision==0.22.1 torchaudio==2.7.1 --index-url https://download.pytorch.org/whl/cu118
```

模型训练参数配置文件位于 `src\lerobot\configs\train.py`，如下图所示，可根据需求自行调整。

```python
    dataset: DatasetConfig
    env: envs.EnvConfig | None = None
    policy: PreTrainedConfig | None = None
    # Set `dir` to where you would like to save all of the run outputs. If you run another training session
    # with the same value for `dir` its contents will be overwritten unless you set `resume` to true.
    output_dir: Path | None = None
    job_name: str | None = None
    # Set `resume` to true to resume a previous run. In order for this to work, you will need to make sure
    # `dir` is the directory of an existing run with at least one checkpoint in it.
    # Note that when resuming a run, the default behavior is to use the configuration from the checkpoint,
    # regardless of what's provided with the training command at the time of resumption.
    resume: bool = False
    # `seed` is used for training (eg: model initialization, dataset shuffling)
    # AND for the evaluation environments.
    seed: int | None = 1000
    # Number of workers for the dataloader.
    num_workers: int = 4
    batch_size: int = 8
    steps: int = 100_000
    eval_freq: int = 20_000
    log_freq: int = 200
    save_checkpoint: bool = True
    # Checkpoint is saved every `save_freq` training iterations and after the last training step.
    save_freq: int = 20_000
    use_policy_training_preset: bool = True
    optimizer: OptimizerConfig | None = None
    scheduler: LRSchedulerConfig | None = None
    eval: EvalConfig = field(default_factory=EvalConfig)
    wandb: WandBConfig = field(default_factory=WandBConfig)

```

参数说明：

- `dataset`：数据集配置对象，定义训练使用的数据集
- `env`：环境配置对象，用于定义机器人仿真或真实环境的参数（可选）
- `policy`：预训练模型配置，定义策略网络的架构和参数（可选）
- `output_dir`：训练输出目录路径，存储模型检查点、日志等文件。若使用相同路径再次训练会覆盖内容（除非启用 `resume`）
- `job_name`：任务名称，用于标识当前训练任务（可选）
- `resume`：是否从检查点恢复训练。设为 `True` 时需确保 `output_dir` 中存在至少一个检查点文件
- `seed`：随机种子（默认 1000），用于控制模型初始化、数据集打乱和评估环境的随机性，确保实验可复现
- `num_workers`：数据加载器的工作进程数（默认 4），影响数据加载速度
- `batch_size`：批次大小（默认 8），每次迭代处理的样本数量
- `steps`：总训练步数（默认 100,000）
- `eval_freq`：评估频率（默认每 20,000 步），定期在验证集上评估模型性能
- `log_freq`：日志记录频率（默认每 200 步），控制训练指标的记录间隔
- `save_checkpoint`：是否保存训练检查点（默认 `True`）
- `save_freq`：检查点保存频率（默认每 20,000 步），最后一步训练后也会保存
- `use_policy_training_preset`：是否使用策略训练预设配置（默认 `True`）
- `optimizer`：优化器配置对象，定义学习率、权重衰减等参数（可选）
- `scheduler`：学习率调度器配置，定义学习率衰减策略（可选）
- `eval`：评估配置对象，包含评估相关的详细设置
- `wandb`：Weights & Biases 配置对象，用于实验跟踪和可视化

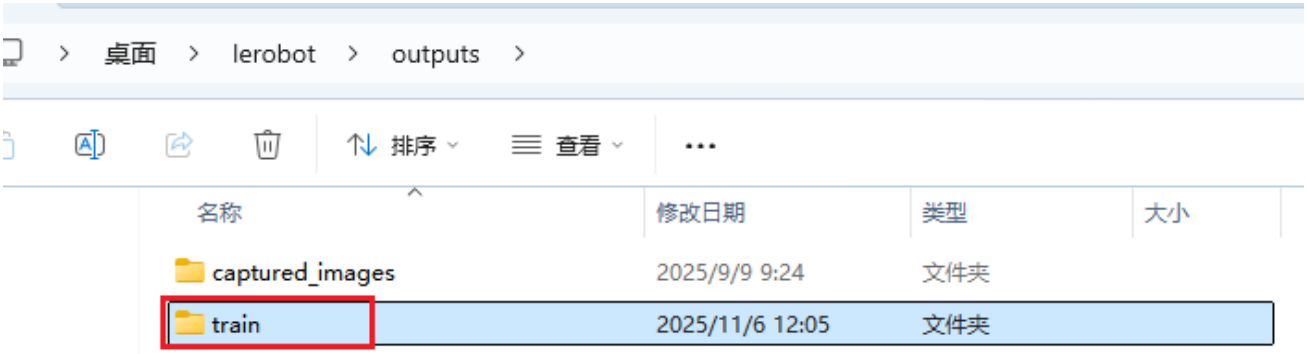
输入指令开始训练。程序开始后会先读取数据集并初始化模型，通常需要等待 1～5 分钟后才会持续显示训练日志。

```
python src/lerobot/scripts/train.py --dataset.repo_id=${HF_USER}/demo --policy.type=act --
output_dir=outputs/train/act_so101_test --job_name=act_so101_test --policy.device=cuda --wandb.enable=false  --
-policy.push_to_hub=false
```

> **Note**
>
> **训练等待时间参考：** 以 RTX4090 和默认 `100000` 步训练为例，通常约需 2～3 小时。显卡性能较低、数据集更大或后台程序占用显存时，训练时间会明显增加；训练过程中请保持电脑通电、避免休眠，并保留足够的磁盘空间用于保存检查点。

```
(lerobot) PS C:\Users\Admin\Desktop\lerobot> python src/lerobot/scripts/train.py --dataset.repo_id=Amdin/demo --policy.t
ype=act --output_dir=outputs/train/act_so101_test --job_name=act_so101_test --policy.device=cuda --wandb.enable=false  -
-policy.push_to_hub=false
```

训练完成后，将在 `lerobot/outputs` 下生成一个 `train` 文件夹，具体路径为 `C:/Users/Admin/Desktop/lerobot/outputs/train`。



7. 模型测试

SO-ARM101 开源 6 轴机械臂 的推理过程采用数据驱动方式，让机器人模仿人类行为。它将复杂的机器人控制问题视为监督学习任务，依赖训练好的模型完成推理。模型会融合环境信息（主要是视觉观察）和自然语言指令，并将其直接映射为控制机器人的具体动作。本质上，这是一种基于“行为克隆”的模仿学习，目标是让机器人通过学习人类演示数据，获得从感知到行动的端到端推理能力，从而完成特定任务。

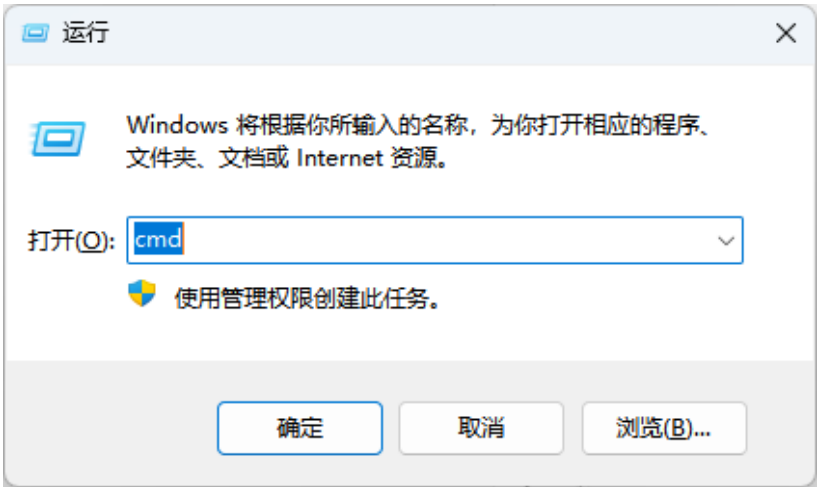
本章使用 `lerobot.record` 命令执行推理，是因为该命令除了加载 `policy.path` 中的模型外，还能统一完成摄像头读取、从臂控制和评估数据保存。推理前应确认模型、摄像头配置和现场摆放与采集训练时保持一致；其中任一项发生变化，都可能导致动作效果下降。

推理部署步骤：

1. 加载训练好的策略模型。
2. 初始化物理机器人及其传感器。
3. 建立传感器数据到策略输入的映射。
4. 建立策略输出到机器人控制指令的映射。
5. 在控制循环中执行策略推理和机器人控制。
6. 可选的实时可视化和监控。

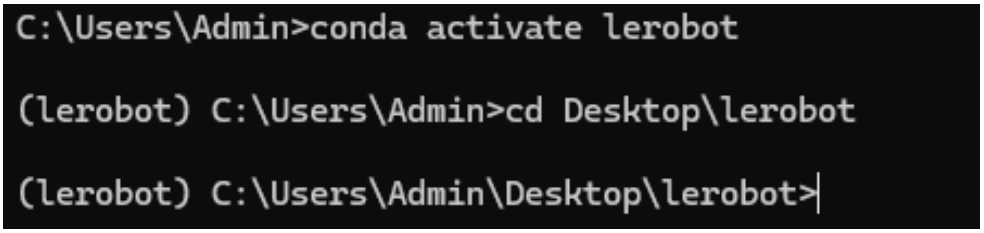
7.1 实时推理测试

按 **Win+R**，输入 **cmd**，打开终端。



输入指令，进入虚拟环境并切换到工程目录。

```
conda activate lerobot
cd Desktop\lerobot
```

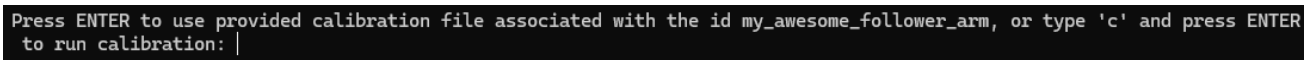


输入指令开始推理。首次启动时会加载模型、连接从臂并初始化摄像头，通常需要等待约 20～60 秒；摄像头较多或 USB 设备刚重新插拔时，初始化时间可能更长。

> **Note**
>
> 1、`outputs/train/act_so101_test/checkpoints/100000/pretrained_model` **这里的** `100000` **为你设置的** `steps` **参数（总训练步数），若想直接使用最后一次训练保存的模型，可将** `"100000"` **改为** `"last"` **即可。**
>
> 2、**推理前请确认摄像头** `ID` **、主臂/从臂端口号、训练模型路径都与当前电脑环境一致。若重新插拔过 USB 设备，建议先重新查找端口号和摄像头** `ID` **。**
>
> 3、`dataset.episode_time_s=60` **表示单次推理或评估的持续时间为 60 秒。若任务执行时间较长，可以适当调大该参数；若只需要短时间测试，也可以调小。**

```
python -m lerobot.record --robot.type=so101_follower --robot.port=COM24 --robot.id=my_awesome_follower_arm --robot.cameras="{ handeye: {type: opencv, index_or_path: 0, width: 640, height: 480, fps: 30}, front: {type: opencv, index_or_path: 1, width: 640, height: 480, fps: 30}}" --display_data=true --dataset.repo_id=${HF_USER}/eval_so101 --dataset.single_task="Grab the screwdriver" --policy.path=outputs/train/act_so101_test/checkpoints/100000/pretrained_model --dataset.episode_time_s=60
```

出现下图提示时，按下 **Enter** 继续。



出现下图标志时，说明机械臂开始推理。

推理开始后，单轮执行时间由 `dataset.episode_time_s` 决定；示例值为 `60`，即每轮约持续 60 秒。推理结束后保存数据通常还需要数秒到几十秒，请等待保存完成提示后再进入下一轮或退出程序。

机械臂到达推理时间后，机械臂将会静止，此时按下 → 保存推理数据，当出现下图标志时，说明本轮推理结束，将会自动进行下一轮的推理。

若要退出推理模式，可以在终端中同时按下“Ctrl”+"c"键。

7.2 推理时常见问题

问题 1：第二次运行推理指令时出现报错

问题描述：在执行推理流程时，首次运行可以正常完成，但在第二次重新执行推理指令时程序报错。系统检测到本地路径 `C:/Users/Admin/Desktop/lerobot/${HF_USER}/eval_so101` 已存在，由于该目录在上一次运行后未被清理或重建，导致新一轮推理任务无法初始化评估环境，从而触发路径冲突错误并中断执行。

根本原因：评估输出目录 `eval_so101` 未实现运行隔离机制或自动清理机制，程序默认不允许覆盖已有文件夹。当第二次运行时，旧的评估结果目录仍然存在，导致目录创建失败或流程初始化异常。

优化建议：执行完推理后，手动将 `eval_so101` 文件夹清理后，再重新执行推理。

问题 2：分布偏移与泛化能力差

问题描述：模型在训练时见过的状态（如物体位置、光照、背景）下表现良好，但一旦遇到与训练数据稍有不同的情况（即“分布外”数据），性能就会明显下降。例如，训练时使用白色桌面，推理时换成木纹桌面，机械臂可能就无法准确识别物体。

根本原因：模型学习到的是训练数据中的状态分布，而不是对物理世界的深层理解。它更接近于“记忆”训练数据，而不是真正“理解”任务。

优化建议：采集数据时尽量覆盖更多物体位置、光照、背景和摆放角度；推理时保持环境与采集环境尽量一致。

问题 3：复合误差

问题描述：这是行为克隆中的经典问题。在推理的每一步，模型都可能产生微小动作误差。由于推理过程通常是开环执行（不会根据结果实时修正），下一步输入状态可能已经受到上一步错误动作影响，并与训练数据中的理想状态越来越远。误差逐步累积后，可能导致机械臂偏离正确轨迹，最终任务失败。

根本原因：推理过程中存在误差积累，当误差持续放大时，机械臂会逐渐偏离原本轨迹。

优化建议：录制数据时保持动作稳定、路径清晰，减少犹豫和重复调整；每一轮尽量从规范的初始姿态开始，避免将异常动作录入数据集。

问题 4：对演示风格的过度拟合

问题描述：模型可能不仅学习了完成任务所需的必要动作，还学习了演示者的个人习惯或特定风格（如低效的抓取姿势）。当环境发生变化时，这种固定风格可能不再有效。

根本原因：模型的目标是最大限度地模仿演示数据，而演示数据中可能包含非最优或与任务无关的行为模式。

优化建议：采集数据时尽量使用稳定、简洁、可重复的动作；如果多人采集，需要统一操作规范，减少风格差异。

8. 常见问题解答

本章汇总使用过程中最常见的问题。遇到报错时，建议先根据问题类型回到对应章节核对环境、端口、校准、摄像头或模型路径，避免在未确认基础配置前反复运行控制命令。

8.1 终端无法识别 `conda`

可能原因是 Miniconda 没有正确加入系统环境变量，或安装后没有重新打开终端。建议先关闭当前终端并重新打开，再执行 `conda env list` 检查。如果仍无法识别，请确认安装 Miniconda 时是否已勾选添加环境变量相关选项。

8.2 无法连接机械臂端口

先检查主臂、从臂 USB 线是否连接稳定，再打开设备管理器确认端口号是否仍为 COM22 和 COM24。如果端口号发生变化，需要在执行命令前同步调整命令中的端口参数。Ubuntu 系统还需要确认 `/dev/ttyACM0`、`/dev/ttyACM1` 等端口是否已授权。

8.3 遥操作时机械臂方向或位置不一致

该问题通常与校准状态有关。请重新将主臂和从臂摆到初始姿态，并分别执行主臂、从臂校准流程。如果之前已经校准过，重新校准时需要按提示输入 `c` 后再继续。

8.4 摄像头画面不对或找不到摄像头

先执行摄像头查找命令，确认 `opencv_0` 和 `opencv_1` 是否都能正常生成画面。若画面顺序与预期不一致，请调整命令中的 `index_or_path`。如果使用拓展坞，建议至少将一个摄像头直接连接电脑 USB 口，不要将两个摄像头同时接入同一个拓展坞。若使用 Ubuntu 系统，还需要确认 `/dev/video*` 等端口是否已授权。

8.5 训练效果不稳定

优先检查数据质量：目标物体位置是否有足够变化、光照是否稳定、画面中是否出现手臂或人员干扰、每轮动作是否完整。数据轮数过少时，模型更容易过拟合，建议增加采集轮数并保持动作规范。

8.6 推理时找不到模型路径

请确认训练输出目录中是否存在 `outputs/train/act_so101_test/checkpoints/100000/pretrained_model`。其中 `100000` 需要与训练时设置的 `steps` 保持一致；如果训练步数不同，请将推理命令中的 checkpoint 路径改为实际生成的目录。

9. 双臂协同操作(拓展)

双臂系统由两条主臂和两条从臂组成：主臂负责人工示教，从臂负责同步执行；在视觉采集、训练和推理阶段，左右两条从臂共同完成双手协同任务。建议按"确认端口 → 设置 ID → 校准 → 无视觉遥操作 → 带视觉遥操作 → 数据采集 → 训练 → 推理"的顺序完成双臂测试。

9.1 端口与环境准备

虚拟环境创建、工程安装和进入工程目录的操作请参考 2.2 环境配置。四条机械臂的端口号查找及固定方法请参考 3.4 查找端口号。

为保持双臂配置参数的命名一致，本章将 **1 号机械臂** 对应为命令中的 `left_arm`，将 **2 号机械臂** 对应为命令中的 `right_arm`。如果实际安装方向相反，请将左右端口参数同时对调，主臂和从臂的对应关系不能交叉。

机械臂	命令中的名称	示例端口号	作用
1 号主臂	`teleop.left_arm_port`	`COM22`	左侧示教输入
2 号主臂	`teleop.right_arm_port`	`COM23`	右侧示教输入
1 号从臂	`robot.left_arm_port`	`COM24`	左侧执行输出
2 号从臂	`robot.right_arm_port`	`COM25`	右侧执行输出

> **Note**
>
> **以下命令以 1 号主臂 `COM22`、2 号主臂 `COM23`、1 号从臂 `COM24`、2 号从臂 `COM25` 为例。实际端口号以设备管理器显示为准，端口发生变化后，需要同步替换后续命令中的 `left_arm_port`、`right_arm_port` 和 `port` 参数。**

```
conda activate lerobot
cd Desktop\lerobot
```

执行本章任意命令前，请确认四条 USB 线均已稳定连接，且终端已经进入 `lerobot` 工程目录。若重新插拔 USB 设备或更换电脑接口，建议先重新确认端口号。

9.2 舵机 ID 设置（选看）

DIY 散件的舵机 ID 设置方法、单个舵机连接要求和装配对应关系请参考 3.5 舵机 ID 设置（选看）。成品机械臂无需执行本节命令。

双臂的 ID 设置仍需逐条机械臂执行，不能将四条机械臂同时接入后一次设置。主臂使用 `so101_leader` 类型，从臂使用 `so101_follower` 类型；每条命令完成后，再切换到下一条机械臂。

左右主臂 ID 设置：

```
lerobot-setup-motors --teleop.type=so101_leader --teleop.port=COM22
lerobot-setup-motors --teleop.type=so101_leader --teleop.port=COM23
```

左右从臂 ID 设置：

```
lerobot-setup-motors --robot.type=so101_follower --robot.port=COM24
lerobot-setup-motors --robot.type=so101_follower --robot.port=COM25
```

9.3 双臂校准

校准前的初始姿态、重新校准时输入 `c` 的操作和关节转动方法请参考 4.1 机械臂校准。

双臂校准命令会依次引导校准两条机械臂。建议先完成左右主臂校准，再完成左右从臂校准；校准时，1 号主臂与 1 号从臂、2 号主臂与 2 号从臂应保持相同的关节对应关系。校准文件与命令中的 `bimanual_leader`、`bimanual_follower` 标识关联，后续遥操作、采集和推理应保持不变。

左右主臂校准：

```
lerobot-calibrate --teleop.type=bi_so101_leader --teleop.left_arm_port=COM22 --teleop.right_arm_port=COM23 --
teleop.id=bimanual_leader
```

左右从臂校准：

```
lerobot-calibrate --robot.type=bi_so101_follower --robot.left_arm_port=COM24 --robot.right_arm_port=COM25 --
robot.id=bimanual_follower
```

9.4 双臂遥操作不带视觉

启动、确认和退出遥操作的操作请参考 4.2 遥操作不带视觉。

无视觉遥操作是双臂首次通电后的推荐检查方式：它不依赖摄像头配置，便于确认主从臂方向、端口和校准结果是否正确。运行后，1 号主臂控制 1 号从臂，2 号主臂控制 2 号从臂；若发现方向、幅度或初始位置异常，应退出程序并重新检查校准，而不要继续进行数据采集。

```
python -m lerobot.teleoperate --robot.type=bi_so101_follower --robot.left_arm_port=COM24 --
robot.right_arm_port=COM25 --robot.id=bimanual_follower --teleop.type=bi_so101_leader --
teleop.left_arm_port=COM22 --teleop.right_arm_port=COM23 --teleop.id=bimanual_leader
```

9.5 双臂遥操作带视觉

摄像头连接、画面确认和退出操作请参考 4.3 遥操作带视觉。双臂默认使用左腕、右腕和环境共 3 个摄像头；如使用第 4 个侧视摄像头，请使用四摄像头版本命令。

双臂视觉配置中，腕部摄像头用于观察各自夹爪附近的细节，环境摄像头用于观察两条从臂的整体协作。先运行摄像头查找命令，根据生成的画面确认每个 `index_or_path` 是否与下表一致，再选择对应的遥操作命令。

摄像头名称	推荐视角	示例 `index_or_path`	三摄像头	四摄像头
`left_wrist`	1 号从臂夹爪	`0`	√	√
`right_wrist`	2 号从臂夹爪	`1`	√	√
`front`	覆盖双臂工作区的环境视角	`2`	√	√
`side`	补充侧面或俯视环境视角	`3`		√

```
lerobot-find-cameras opencv
```

三摄像头版本：

```
python -m lerobot.teleoperate --robot.type=bi_so101_follower --robot.left_arm_port=COM24 --
robot.right_arm_port=COM25 --robot.id=bimanual_follower --robot.cameras="{ left_wrist: {type: opencv,
index_or_path: 0, width: 640, height: 480, fps: 30}, right_wrist: {type: opencv, index_or_path: 1, width: 640,
height: 480, fps: 30}, front: {type: opencv, index_or_path: 2, width: 640, height: 480, fps: 30}}" --
teleop.type=bi_so101_leader --teleop.left_arm_port=COM22 --teleop.right_arm_port=COM23 --
teleop.id=bimanual_leader --display_data=true
```

四摄像头版本：

```
python -m lerobot.teleoperate --robot.type=bi_so101_follower --robot.left_arm_port=COM24 --
robot.right_arm_port=COM25 --robot.id=bimanual_follower --robot.cameras="{ left_wrist: {type: opencv,
index_or_path: 0, width: 640, height: 480, fps: 30}, right_wrist: {type: opencv, index_or_path: 1, width: 640,
height: 480, fps: 30}, front: {type: opencv, index_or_path: 2, width: 640, height: 480, fps: 30}, side: {type:
opencv, index_or_path: 3, width: 640, height: 480, fps: 30}}" --teleop.type=bi_so101_leader --
teleop.left_arm_port=COM22 --teleop.right_arm_port=COM23 --teleop.id=bimanual_leader --display_data=true
```

> **Note**
>
> **`left_wrist`、`right_wrist`、`front` 和 `side` 会成为数据集特征名称。数据采集、训练和推理时，摄像头数量、名称及其 `index_or_path` 对应关系必须保持一致。**

9.6 双臂数据采集

录制、保存和复位数据的操作请参考 5. 数据采集。`${HF_USER}` 可替换为自己的英文用户名，`dataset.num_episodes=20` 可按实际采集轮数修改。

双臂数据需要体现两条机械臂之间的配合关系，例如一条机械臂固定物体、另一条机械臂完成操作。每轮任务描述 `dataset.single_task`、双臂初始姿态和摄像头配置应尽量一致；为提升泛化能力，可在不改变任务流程的前提下变化目标物的位置和朝向。

参数	作用	使用建议
`dataset.repo_id`	数据集保存名称	三摄像头使用 `demo`，四摄像头使用 `demo_4cam`
`dataset.num_episodes`	录制轮数	可由 `20` 改为实际需要的轮数
`dataset.single_task`	任务文字描述	同一数据集内保持相同描述
`dataset.push_to_hub`	是否上传至 Hub	本地采集保持 `false`

三摄像头版本：

```
python -m lerobot.record --robot.type=bi_so101_follower --robot.left_arm_port=COM24 --
robot.right_arm_port=COM25 --robot.id=bimanual_follower --robot.cameras="{ left_wrist: {type: opencv,
index_or_path: 0, width: 640, height: 480, fps: 30}, right_wrist: {type: opencv, index_or_path: 1, width: 640,
height: 480, fps: 30}, front: {type: opencv, index_or_path: 2, width: 640, height: 480, fps: 30}}" --
teleop.type=bi_so101_leader --teleop.left_arm_port=COM22 --teleop.right_arm_port=COM23 --
teleop.id=bimanual_leader --display_data=true --dataset.repo_id=${HF_USER}/demo --dataset.num_episodes=20 --
dataset.single_task="Grab the screwdriver with both arms" --dataset.push_to_hub=false
```

四摄像头版本：

```
python -m lerobot.record --robot.type=bi_so101_follower --robot.left_arm_port=COM24 --
robot.right_arm_port=COM25 --robot.id=bimanual_follower --robot.cameras="{ left_wrist: {type: opencv,
index_or_path: 0, width: 640, height: 480, fps: 30}, right_wrist: {type: opencv, index_or_path: 1, width: 640,
height: 480, fps: 30}, front: {type: opencv, index_or_path: 2, width: 640, height: 480, fps: 30}, side: {type:
opencv, index_or_path: 3, width: 640, height: 480, fps: 30}}" --teleop.type=bi_so101_leader --
teleop.left_arm_port=COM22 --teleop.right_arm_port=COM23 --teleop.id=bimanual_leader --display_data=true --
dataset.repo_id=${HF_USER}/demo_4cam --dataset.num_episodes=20 --dataset.single_task="Grab the screwdriver
with both arms" --dataset.push_to_hub=false
```

9.7 双臂数据集训练

云服务器训练、本机训练及训练依赖安装请参考 6. 数据集训练。训练命令会从数据集中读取摄像头特征，因此不需要额外填写 `robot.cameras` 参数。三摄像头与四摄像头数据集需要分别训练，不可混用。

9.7.1 云服务器训练

云服务器的租用、数据集上传、工程包上传和 `screen` 会话使用方法请参考 6.1 云服务器训练。执行下列命令前，请确认已将数据集放入云服务器的 LeRobot 缓存目录；云服务器示例直接使用 `demo` 或 `demo_4cam` 作为数据集名称。

数据配置	数据集名称	输出目录	后续推理模型目录
三摄像头	`demo`	`outputs/train/act_so101_test`	`act_so101_test`
四摄像头	`demo_4cam`	`outputs/train/act_so101_4cam`	`act_so101_4cam`

三摄像头数据集：

```
python lerobot/scripts/train.py --dataset.repo_id=demo --policy.type=act --
output_dir=outputs/train/act_so101_test --job_name=act_so101_test --policy.device=cuda --wandb.enable=false --
policy.push_to_hub=false
```

四摄像头数据集：

```
python lerobot/scripts/train.py --dataset.repo_id=demo_4cam --policy.type=act --
output_dir=outputs/train/act_so101_4cam --job_name=act_so101_4cam --policy.device=cuda --wandb.enable=false --
policy.push_to_hub=false
```

9.7.2 本机训练

本机训练的显卡要求、PyTorch 安装和训练环境准备请参考 6.2 本机训练。本机数据集默认位于当前 Windows 用户目录的缓存路径中，因此命令中的 `${HF_USER}` 需要替换为采集数据时使用的英文用户名。

数据配置	数据集名称	输出目录	后续推理模型目录
三摄像头	`${HF_USER}/demo`	`outputs/train/act_so101_test`	`act_so101_test`
四摄像头	`${HF_USER}/demo_4cam`	`outputs/train/act_so101_4cam`	`act_so101_4cam`

三摄像头数据集：

```
python src/lerobot/scripts/train.py --dataset.repo_id=${HF_USER}/demo --policy.type=act --
output_dir=outputs/train/act_so101_test --job_name=act_so101_test --policy.device=cuda --wandb.enable=false --
policy.push_to_hub=false
```

四摄像头数据集：

```
python src/lerobot/scripts/train.py --dataset.repo_id=${HF_USER}/demo_4cam --policy.type=act --
output_dir=outputs/train/act_so101_4cam --job_name=act_so101_4cam --policy.device=cuda --wandb.enable=false --
policy.push_to_hub=false
```

9.8 双臂模型推理

推理启动、模型路径确认和结果保存操作请参考 7.1 实时推理测试。命令中的 `100000` 需与训练时设置的 `steps` 保持一致；如需加载最后一次保存的模型，可将其替换为 `last`。

推理时仅连接两条从臂和摄像头即可，主臂无需参与控制。程序会读取训练好的策略模型，并根据双腕与环境画面同时输出两条从臂的动作。首次测试建议将 `dataset.episode_time_s` 保持为 `60`，待动作稳定后再根据任务时长调整。

参数	作用	注意事项
`policy.path`	已训练模型路径	必须与摄像头版本和实际检查点目录一致
`dataset.repo_id`	推理数据保存名称	每次重新推理建议使用新的名称或清理旧目录
`dataset.single_task`	当前任务描述	建议与采集训练时的任务描述保持一致
`dataset.episode_time_s`	单轮推理持续时间	默认示例为 `60` 秒，可按任务时长修改

三摄像头模型推理：

```
python -m lerobot.record --robot.type=bi_so101_follower --robot.left_arm_port=COM24 --
robot.right_arm_port=COM25 --robot.id=bimanual_follower --robot.cameras="{ left_wrist: {type: opencv,
index_or_path: 0, width: 640, height: 480, fps: 30}, right_wrist: {type: opencv, index_or_path: 1, width: 640,
height: 480, fps: 30}, front: {type: opencv, index_or_path: 2, width: 640, height: 480, fps: 30}}" --
display_data=true --dataset.repo_id=${HF_USER}/eval_so101_bimanual --dataset.single_task="Grab the screwdriver
with both arms" --policy.path=outputs/train/act_so101_test/checkpoints/100000/pretrained_model --
dataset.episode_time_s=60
```

四摄像头模型推理：

```
python -m lerobot.record --robot.type=bi_so101_follower --robot.left_arm_port=COM24 --
robot.right_arm_port=COM25 --robot.id=bimanual_follower --robot.cameras="{ left_wrist: {type: opencv,
index_or_path: 0, width: 640, height: 480, fps: 30}, right_wrist: {type: opencv, index_or_path: 1, width: 640,
height: 480, fps: 30}, front: {type: opencv, index_or_path: 2, width: 640, height: 480, fps: 30}, side: {type:
opencv, index_or_path: 3, width: 640, height: 480, fps: 30}}" --display_data=true --
dataset.repo_id=${HF_USER}/eval_so101_bimanual_4cam --dataset.single_task="Grab the screwdriver with both
arms" --policy.path=outputs/train/act_so101_4cam/checkpoints/100000/pretrained_model --
dataset.episode_time_s=60
```